

Piecewise-Defined Functions Domain

Abstract Domain Operators

- They manipulate elements in $\mathcal{A}_{\text{NIL}} \stackrel{\text{def}}{=} \{\text{NIL}\} \cup \mathcal{A}$
- The **binary operators** rely on a tree unification algorithm
 - approximation order $\preceq_A$ and computational order $\sqsubseteq_A$
 - approximation join $\vee_A$ and computational join $\sqcup_A$
 - meet $\wedge_A$
 - widening ∇_A
- The **unary operators** rely on a tree pruning algorithm
 - assignment $\overleftarrow{\text{ASSIGN}}_A[[X \leftarrow e]]$
 - test $\text{FILTER}_A[[e]]$

Piecewise-Defined Functions Domain

Abstract Domain Operators

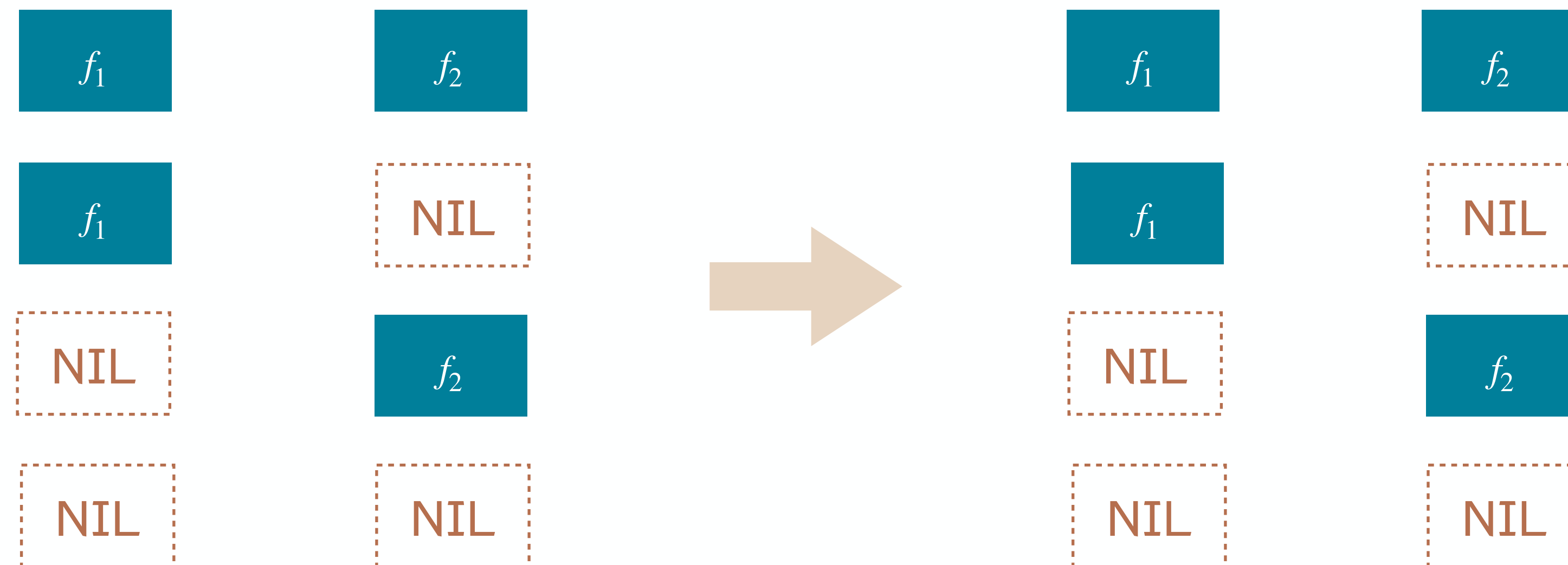
- They manipulate elements in $\mathcal{A}_{\text{NIL}} \stackrel{\text{def}}{=} \{\text{NIL}\} \cup \mathcal{A}$
- The **binary operators** rely on a tree unification algorithm
 - approximation order $\preceq_A$ and computational order $\sqsubseteq_A$
 - approximation join $\vee_A$ and computational join $\sqcup_A$
 - meet $\wedge_A$
 - widening ∇_A
- The **unary operators** rely on a tree pruning algorithm
 - assignment $\overleftarrow{\text{ASSIGN}}_A[[X \leftarrow e]]$
 - test $\text{FILTER}_A[[e]]$

Piecewise-Defined Functions Domain

Tree Unification

Goal: find a **common refinement** for the given decision trees

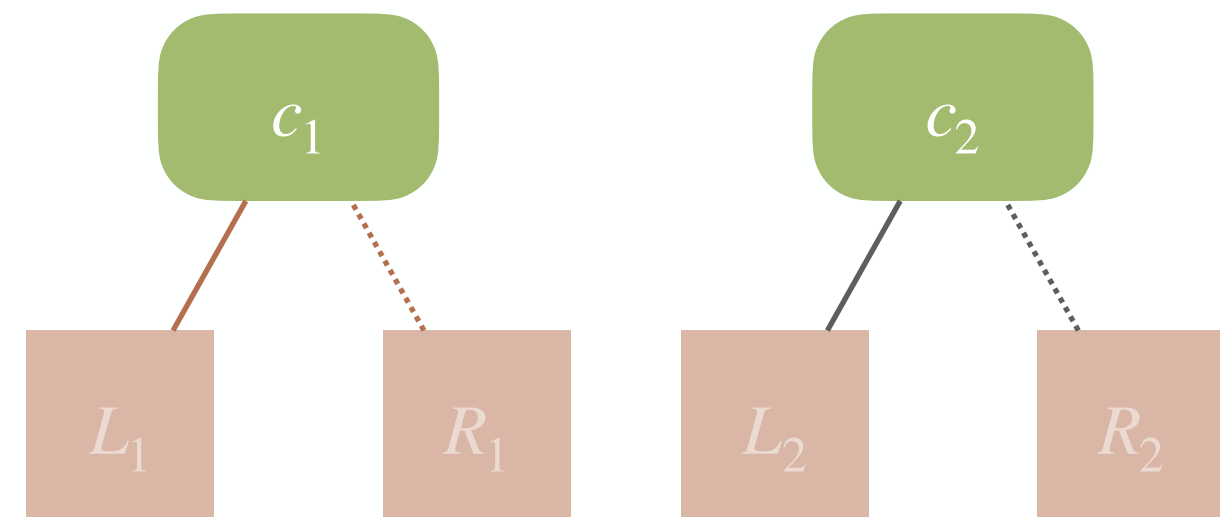
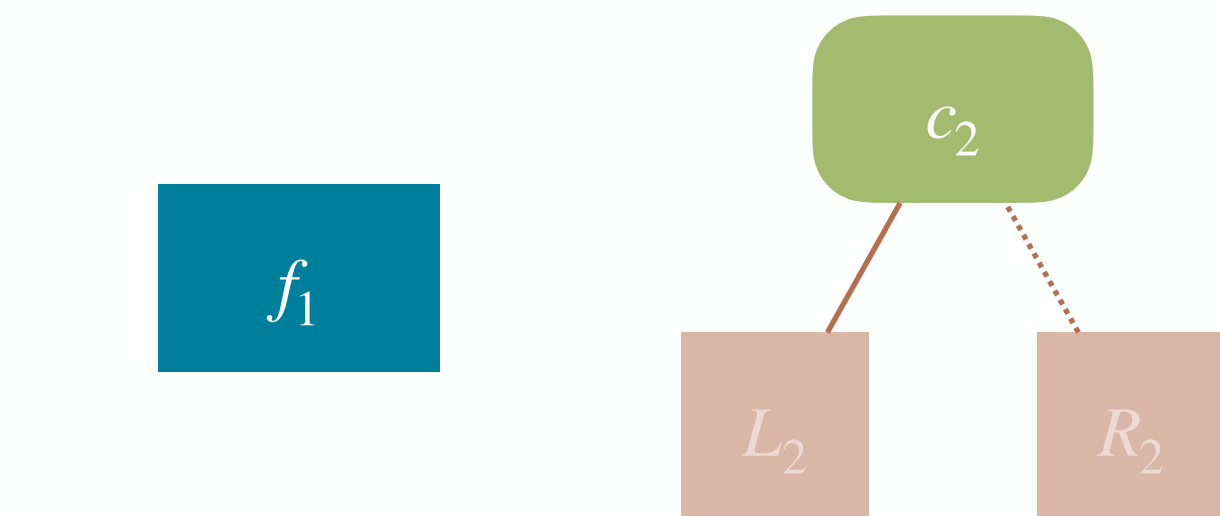
- Base cases:



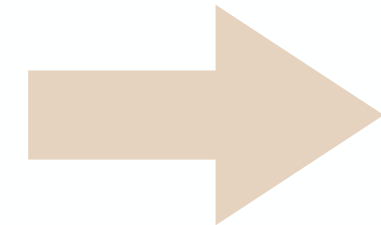
Piecewise-Defined Functions Domain

Tree Unification (continue)

- Case ①



$$c_2 \leq_C c_1$$



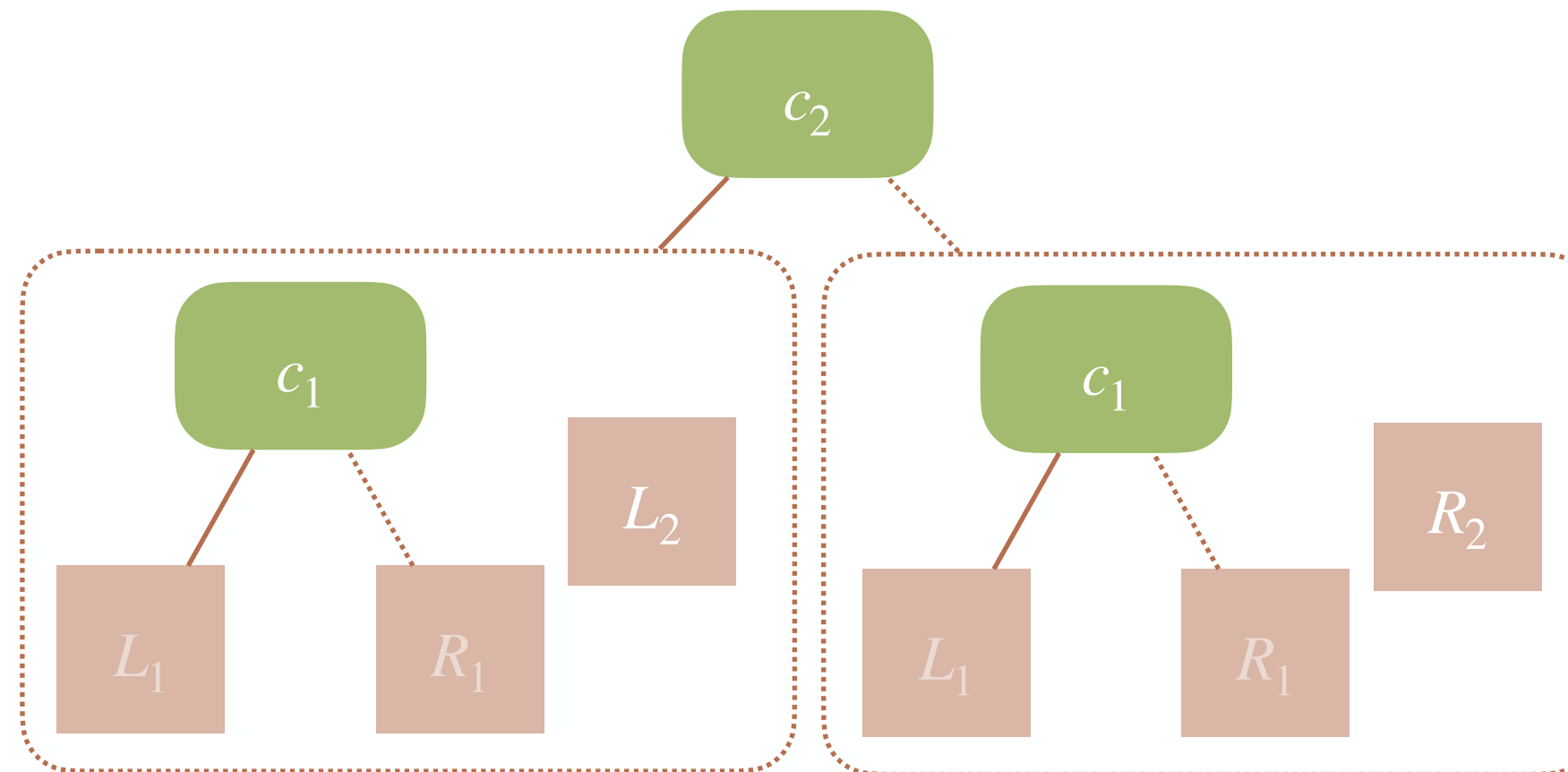
①a) c_2 is redundant



①b) $\neg c_2$ is redundant



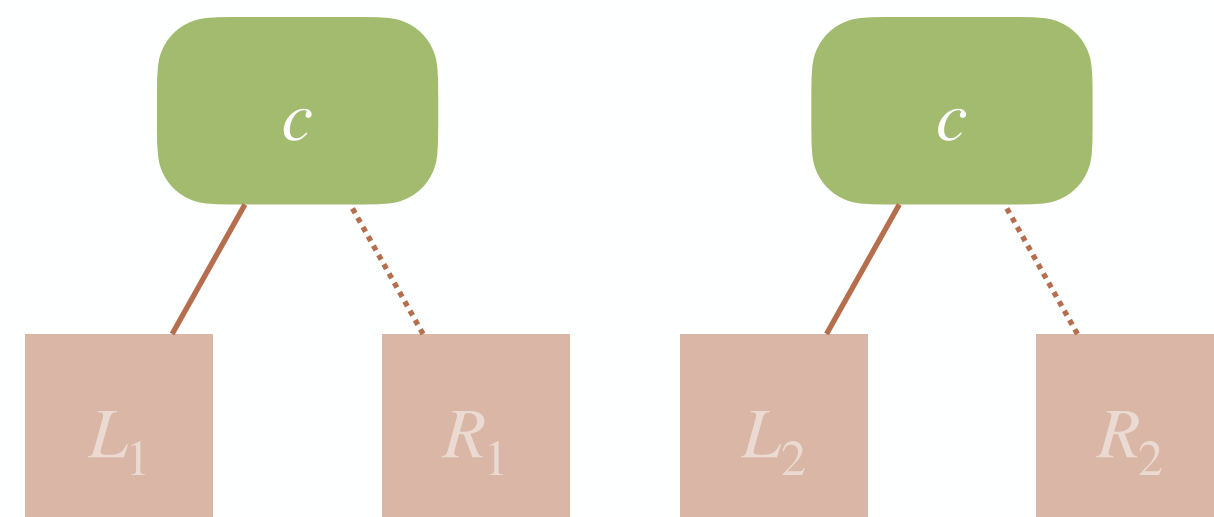
①c) c_2 is added to t_1



Piecewise-Defined Functions Domain

Tree Unification (continue)

- Case ② (simmetric to ①)
- Case ③



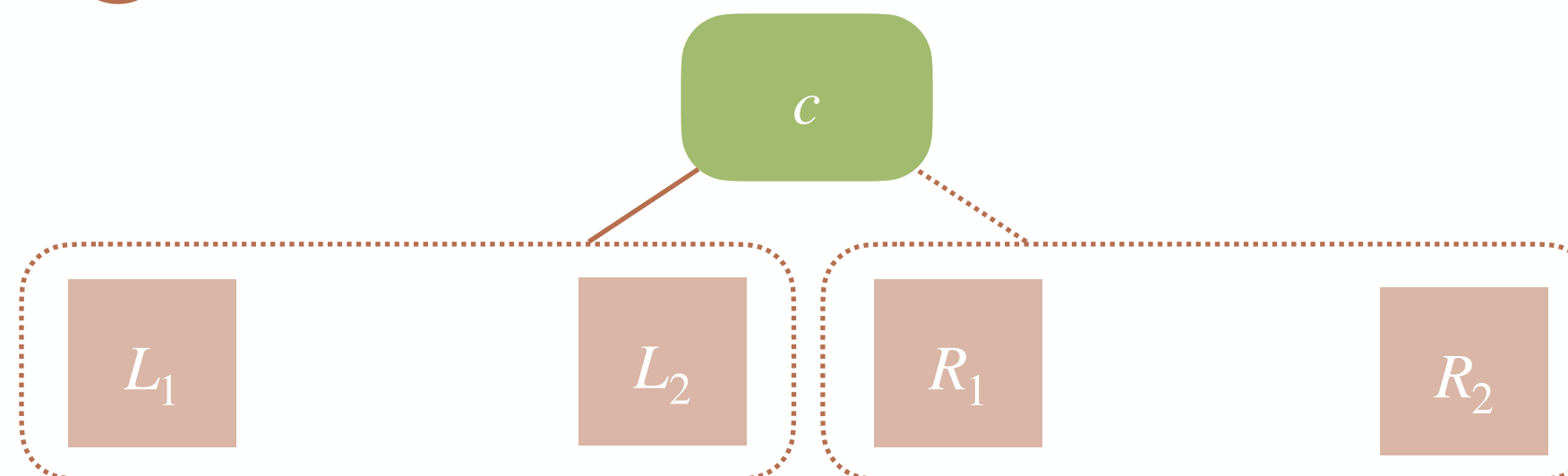
①a) c is redundant



①b) $\neg c$ is redundant



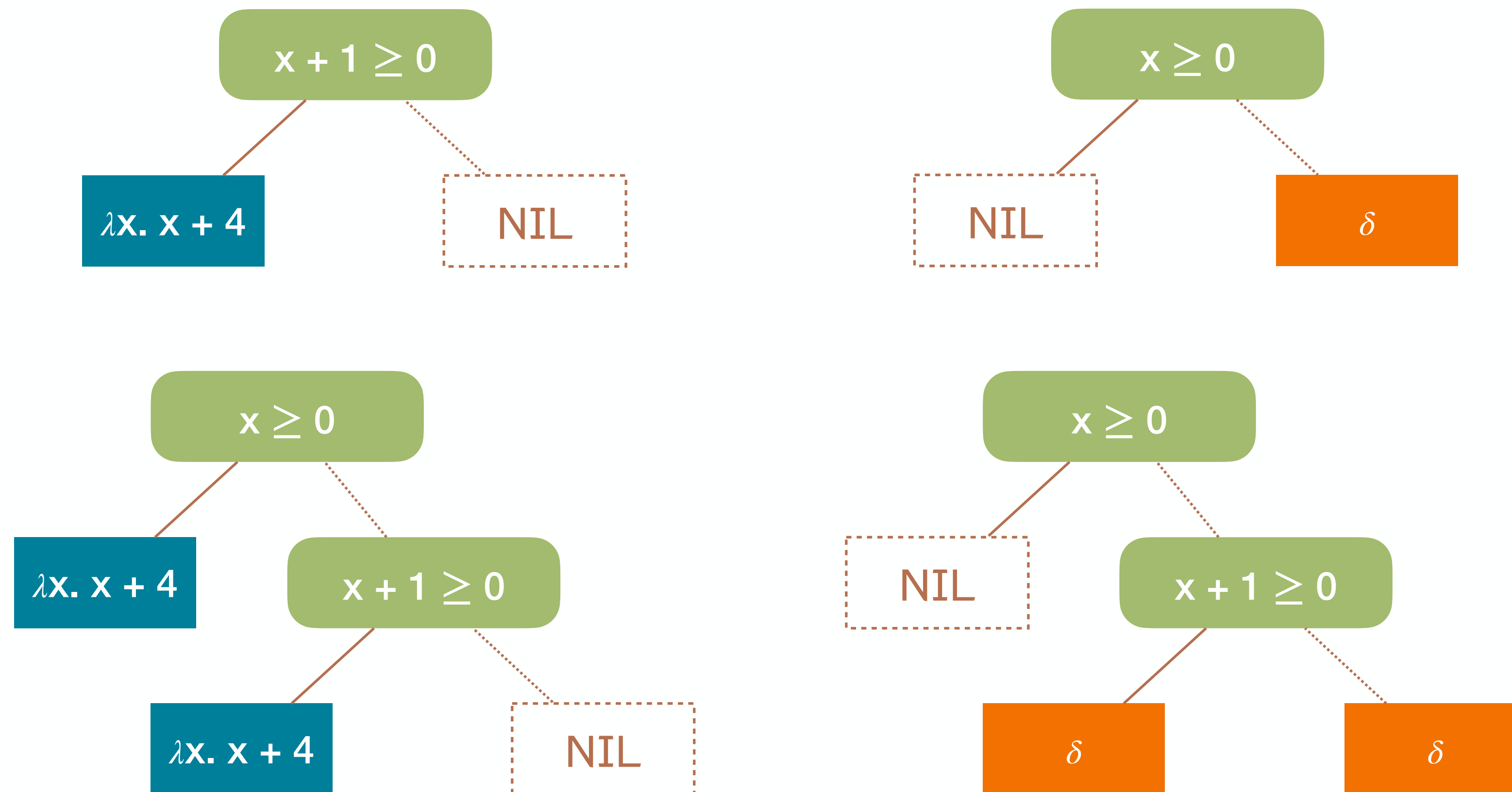
①c) c is kept in t_1 and t_2



Piecewise-Defined Functions Domain

Tree Unification (continue)

Example



Piecewise-Defined Functions Domain

Abstract Domain Operators

- They manipulate elements in $\mathcal{A}_{\text{NIL}} \stackrel{\text{def}}{=} \{\text{NIL}\} \cup \mathcal{A}$
- The **binary operators** rely on a tree unification algorithm
 - **approximation order** $\preceq_A$ and **computational order** $\sqsubseteq_A$
 - approximation join $\vee_A$ and computational join $\sqcup_A$
 - meet $\wedge_A$
 - widening ∇_A
- The **unary operators** rely on a tree pruning algorithm
 - assignment $\overleftarrow{\text{ASSIGN}}_A[[X \leftarrow e]]$
 - test $\text{FILTER}_A[[e]]$

Piecewise-Defined Functions Domain

Order

1. Perform **tree unification**
2. Recursively descend the trees while *accumulating the linear constraints encountered along the paths* into a set of constraints C
3. Compare the leaf nodes using the **approximation order** $\preceq_F[\alpha_C(C)]$ or the **computational order** $\sqsubseteq_F[\alpha_C(C)]$

The concretization function γ_A is monotonic with respect to $\preceq_A$:

Lemma

$$\forall t_1, t_2 \in \mathcal{A} : t_1 \preceq_A t_2 \Rightarrow \gamma_A(t_1) \preceq \gamma_A(t_2)$$

Piecewise-Defined Functions Domain

Abstract Domain Operators

- They manipulate elements in $\mathcal{A}_{\text{NIL}} \stackrel{\text{def}}{=} \{\text{NIL}\} \cup \mathcal{A}$
- The **binary operators** rely on a tree unification algorithm
 - approximation order $\preceq_A$ and computational order $\sqsubseteq_A$
 - **approximation join** $\vee_A$ and **computational join** $\sqcup_A$
 - meet $\wedge_A$
 - widening ∇_A
- The **unary operators** rely on a tree pruning algorithm
 - assignment $\overleftarrow{\text{ASSIGN}}_A[[X \leftarrow e]]$
 - test $\text{FILTER}_A[[e]]$

Piecewise-Defined Functions Domain

Join

1. Perform **tree unification**
2. Recursively descend the trees while *accumulating the linear constraints encountered along the paths* into a set of constraints C
3.
$$\begin{array}{l} \text{NIL} \vee_A t \stackrel{\text{def}}{=} t \\ t \vee_A \text{NIL} \stackrel{\text{def}}{=} t \end{array}$$
4. Join the leaf nodes using
the **approximation join** $\vee_F [\alpha_C(C)]$
or the **computational join** $\sqcup_F [\alpha_C(C)]$

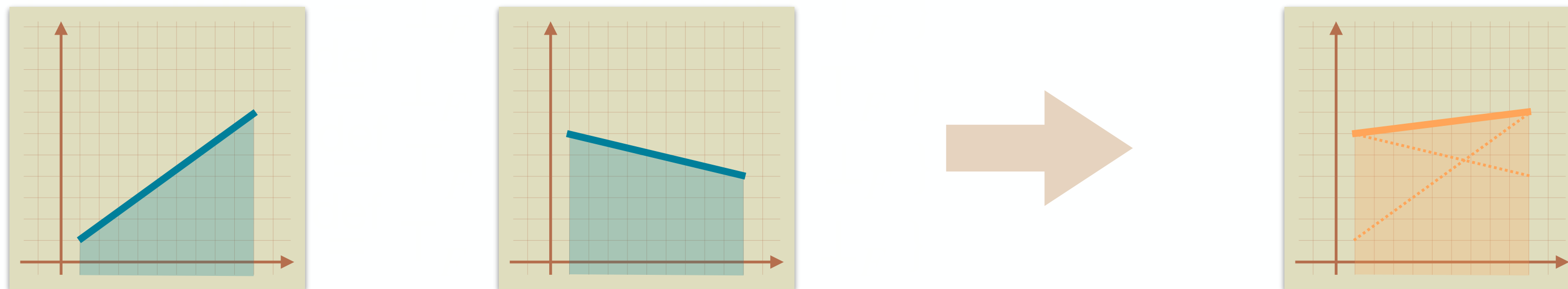
Piecewise-Defined Functions Domain

Join (continue)

- **approximation join** $\vee_F [D]$, where $D \in \mathcal{D}$:
- between defined leaf nodes:

$$f_1 \vee_F [D] f_2 \stackrel{\text{def}}{=} \begin{cases} f = \lambda \rho \in \gamma_D(D) : \max(f_1(\dots, \rho(X_i), \dots), f_2(\dots, \rho(X_i), \dots)) & f \in \mathcal{F} \setminus \{ \perp_F, \top_F \} \\ \top_F & \text{otherwise} \end{cases}$$

Example:



Piecewise-Defined Functions Domain

Join (continue)

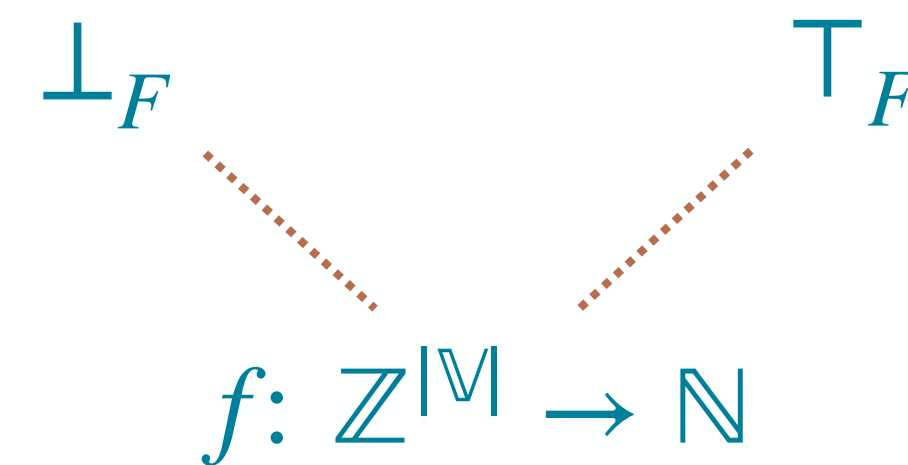
- **approximation join** $\vee_F [D]$, where $D \in \mathcal{D}$:

- between defined leaf nodes:

$$f_1 \vee_F [D] f_2 \stackrel{\text{def}}{=} \begin{cases} f = \lambda \rho \in \gamma_D(D) : \max(f_1(\dots, \rho(X_i), \dots), f_2(\dots, \rho(X_i), \dots)) & f \in \mathcal{F} \setminus \{ \perp_F, \top_F \} \\ \top_F & \text{otherwise} \end{cases}$$

- otherwise (i.e., when one or both leaf nodes are undefined):

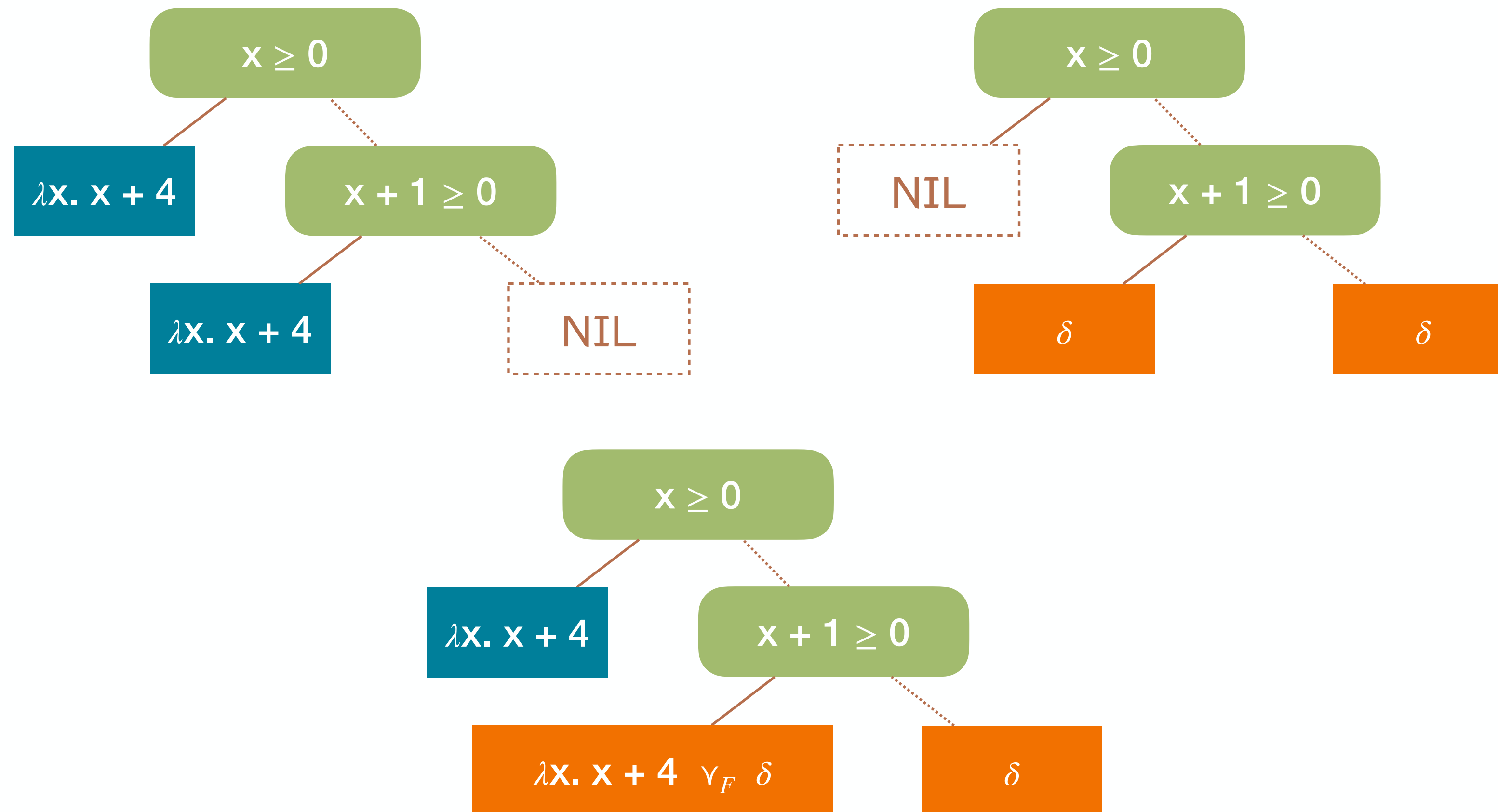
$$\begin{array}{ll} \perp_F \vee_F [D] f \stackrel{\text{def}}{=} \perp_F & f \in \mathcal{F} \setminus \{ \top_F \} \\ f \vee_F [D] \perp_F \stackrel{\text{def}}{=} \perp_F & f \in \mathcal{F} \setminus \{ \top_F \} \\ \top_F \vee_F [D] f \stackrel{\text{def}}{=} \top_F & f \in \mathcal{F} \setminus \{ \perp_F \} \\ f \vee_F [D] \top_F \stackrel{\text{def}}{=} \top_F & f \in \mathcal{F} \setminus \{ \perp_F \} \end{array}$$



Piecewise-Defined Functions Domain

Join (continue)

Example



Piecewise-Defined Functions Domain

Join (continue)

- **computational join** $\sqcup_F [D]$, where $D \in \mathcal{D}$:

- between defined leaf nodes:

$$f_1 \vee_F [D] f_2 \stackrel{\text{def}}{=} \begin{cases} f = \lambda \rho \in \gamma_D(D) : \max(f_1(\dots, \rho(X_i), \dots), f_2(\dots, \rho(X_i), \dots)) & f \in \mathcal{F} \setminus \{ \perp_F, \top_F \} \\ \top_F & \text{otherwise} \end{cases}$$

- otherwise (i.e., when one or both leaf nodes are undefined):

$$\begin{aligned} \perp_F \sqcup_F [D] f &\stackrel{\text{def}}{=} f & f \in \mathcal{F} \\ f \sqcup_F [D] \perp_F &\stackrel{\text{def}}{=} f & f \in \mathcal{F} \\ \top_F \sqcup_F [D] f &\stackrel{\text{def}}{=} \top_F & f \in \mathcal{F} \\ f \sqcup_F [D] \top_F &\stackrel{\text{def}}{=} \top_F & f \in \mathcal{F} \end{aligned}$$

$$\begin{array}{c} \top_F \\ \vdots \\ f: \mathbb{Z}^{\mathbb{N}} \rightarrow \mathbb{N} \\ \vdots \\ \perp_F \end{array}$$

Piecewise-Defined Functions Domain

Abstract Domain Operators

- They manipulate elements in $\mathcal{A}_{\text{NIL}} \stackrel{\text{def}}{=} \{\text{NIL}\} \cup \mathcal{A}$
- The **binary operators** rely on a tree unification algorithm
 - approximation order $\preceq_A$ and computational order $\sqsubseteq_A$
 - approximation join $\vee_A$ and computational join $\sqcup_A$
 - **meet** $\wedge_A$
 - widening ∇_A
- The **unary operators** rely on a tree pruning algorithm
 - assignment $\overleftarrow{\text{ASSIGN}}_A[[X \leftarrow e]]$
 - test $\text{FILTER}_A[[e]]$

Piecewise-Defined Functions Domain

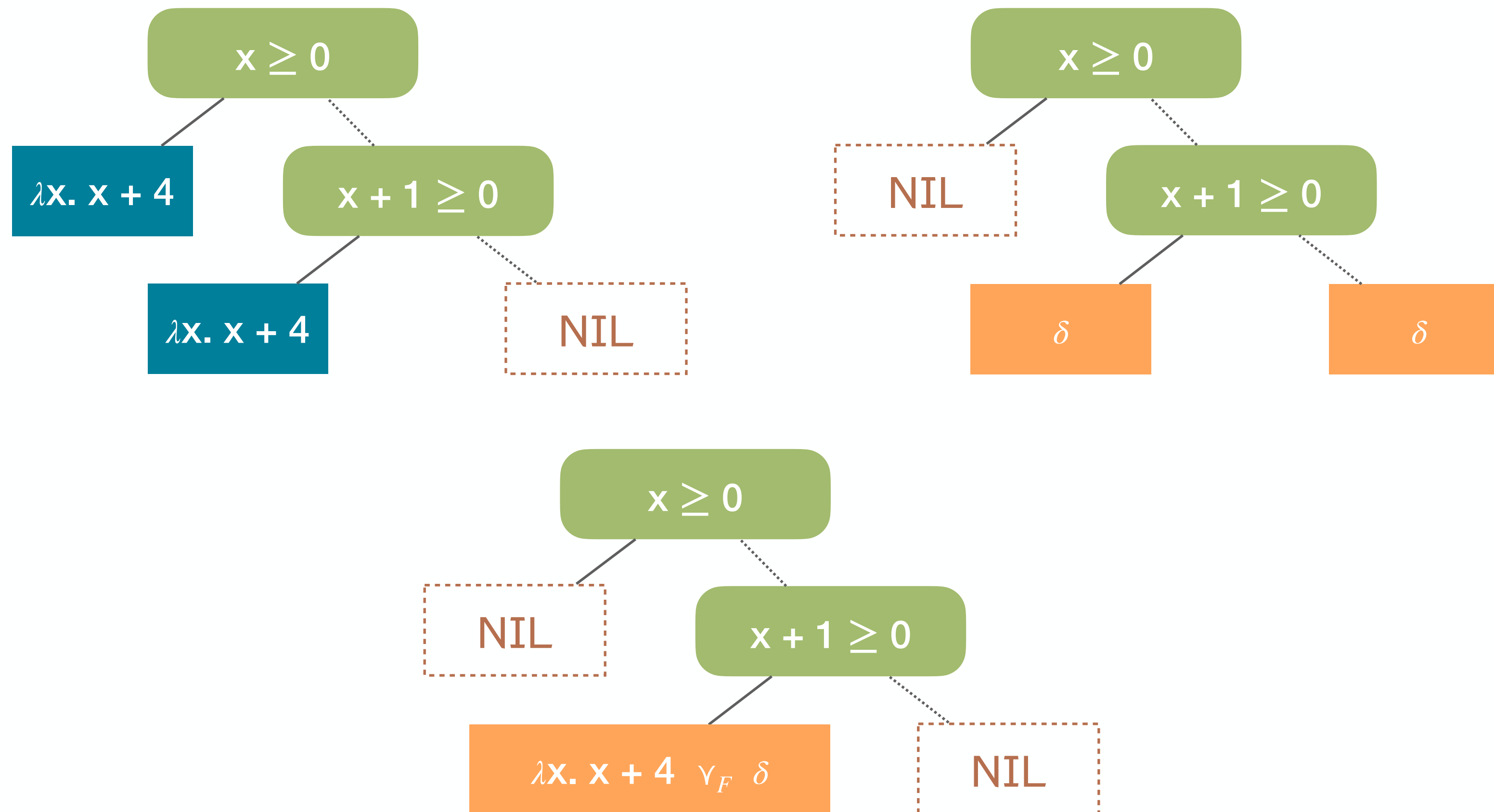
Meet

1. Perform **tree unification**
2. Recursively descend the trees while *accumulating the linear constraints encountered along the paths* into a set of constraints C
3.
$$\begin{aligned} \text{NIL} \vee_A t &\stackrel{\text{def}}{=} \text{NIL} \\ t \vee_A \text{NIL} &\stackrel{\text{def}}{=} \text{NIL} \end{aligned}$$
4. Join the leaf nodes using the **approximation join** $\vee_F [\alpha_C(C)]$

Piecewise-Defined Functions Domain

Meet (continue)

Example



Piecewise-Defined Functions Domain

Abstract Domain Operators

- They manipulate elements in $\mathcal{A}_{\text{NIL}} \stackrel{\text{def}}{=} \{\text{NIL}\} \cup \mathcal{A}$
- The **binary operators** rely on a tree unification algorithm
 - approximation order $\preceq_A$ and computational order $\sqsubseteq_A$
 - approximation join $\vee_A$ and computational join $\sqcup_A$
 - meet $\wedge_A$
 - **widening** ∇_A
- The **unary operators** rely on a tree pruning algorithm
 - assignment $\overleftarrow{\text{ASSIGN}}_A[[X \leftarrow e]]$
 - test $\text{FILTER}_A[[e]]$

Piecewise-Defined Functions Domain

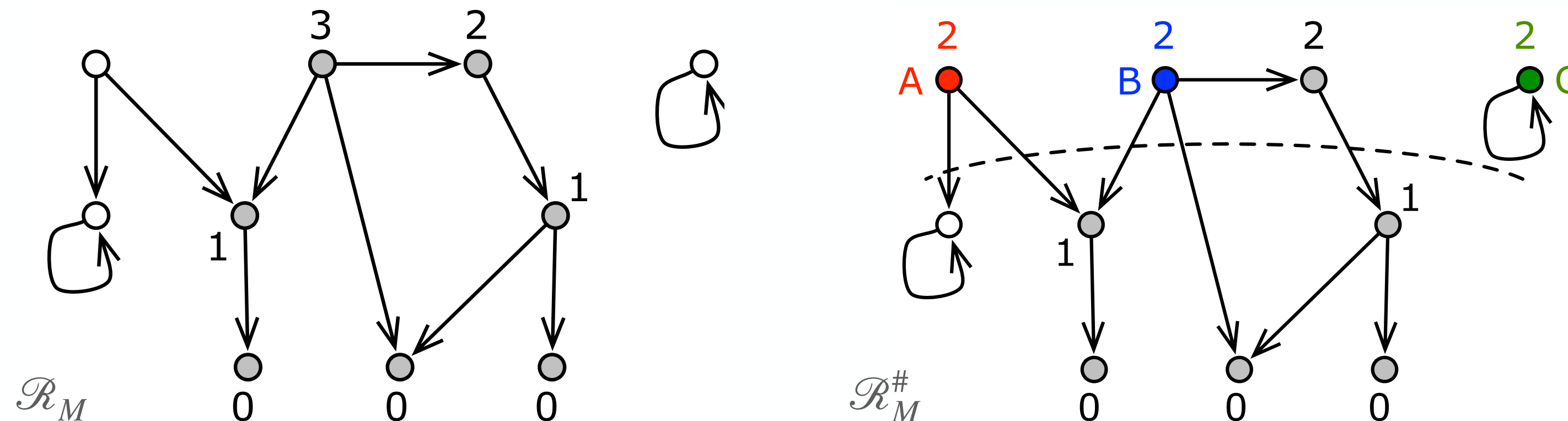
Widening

Goal: try to **predict** a valid ranking function

The prediction can (temporarily) be wrong!, i.e., it can

- *under-approximate* the value of $\mathcal{R}_M$
- and/or
- *over-approximate* the domain $\text{dom}(\mathcal{R}_M)$ of $\mathcal{R}_M$

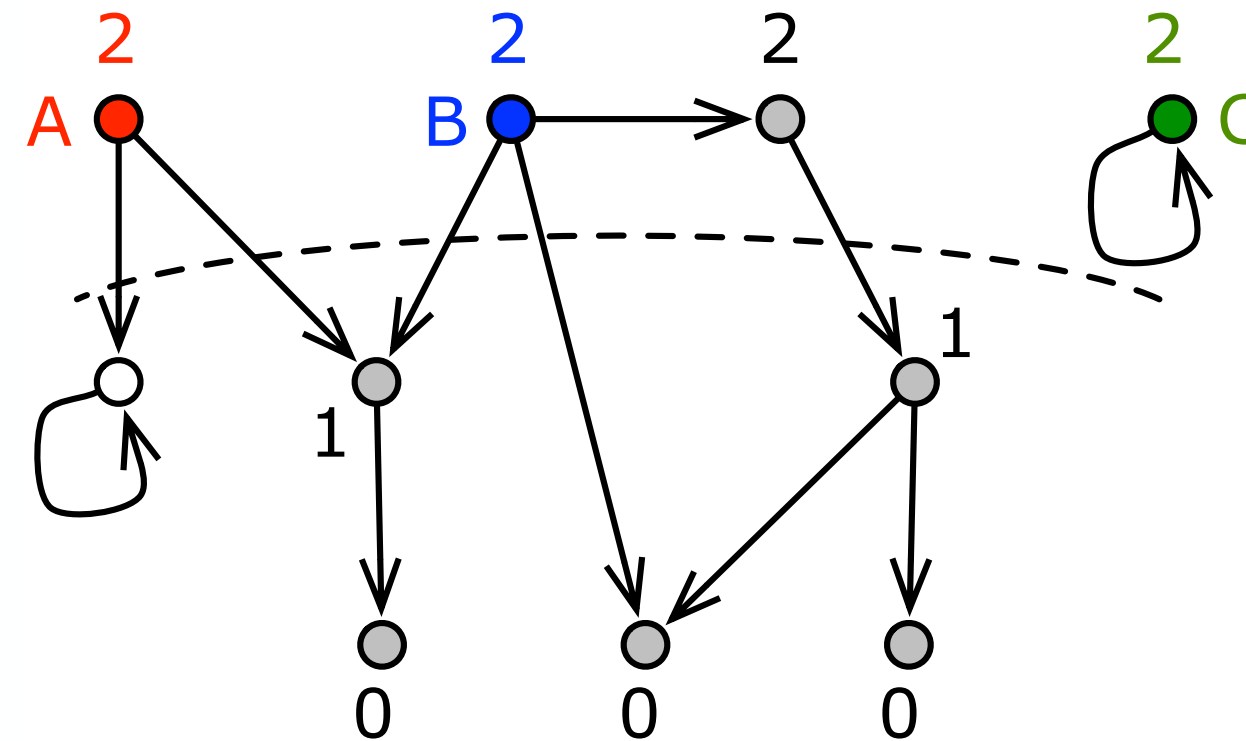
Example



Piecewise-Defined Functions Domain

Widening (continue)

1. Check for **case A** (i.e., wrong domain predictions)
2. Perform **domain widening**
3. Check for **case B or C** (i.e., wrong value predictions)
4. Perform **value widening**



Piecewise-Defined Functions Domain

Widening (continue)

Check for Case A

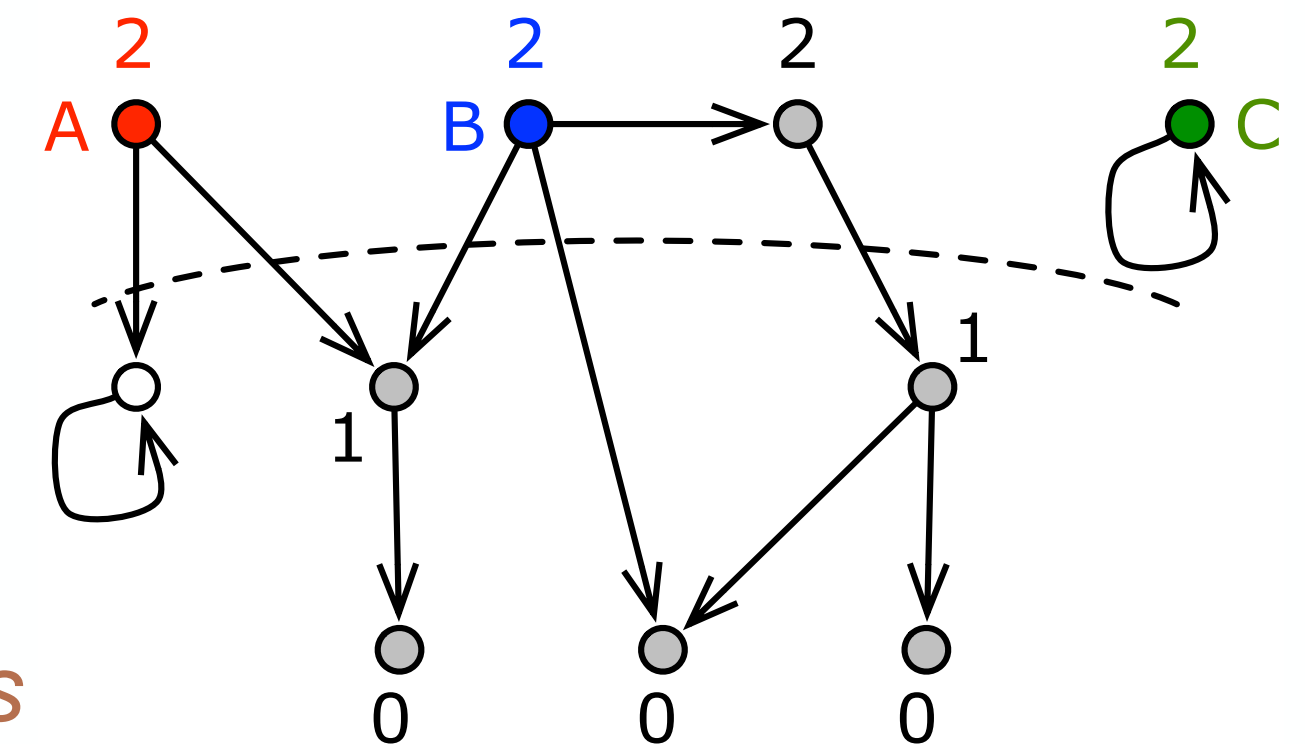
Lemma

Let $\text{dom}(\gamma_A(\mathcal{R}_M^{\#n}(\ell))) \setminus \text{dom}(\mathcal{R}_M(\ell)) \neq \emptyset$. Then, in case A, we have $\text{dom}(\gamma_A(\mathcal{R}_M^{\#n+1}(\ell))) \setminus \text{dom}(\mathcal{R}_M(\ell)) \subset \text{dom}(\gamma_A(\mathcal{R}_M^{\#n}(\ell))) \setminus \text{dom}(\mathcal{R}_M(\ell))$.

1. Perform **tree unification**

2. Recursively descend the trees while *accumulating the linear constraints encountered along the paths* into a set of constraints C

3. f $\perp_F$ $\rightarrow$ f $\top_F$



Piecewise-Defined Functions Domain

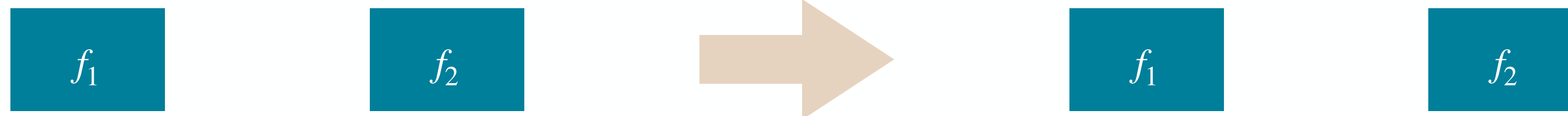
Widening (continue)

Domain Widening

Goal: **limit the size** of the decision trees

Left unification: variant of tree unification that forces the structure of t_1 on t_2

- Base case:

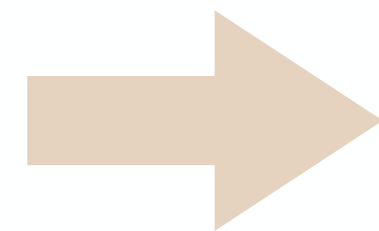
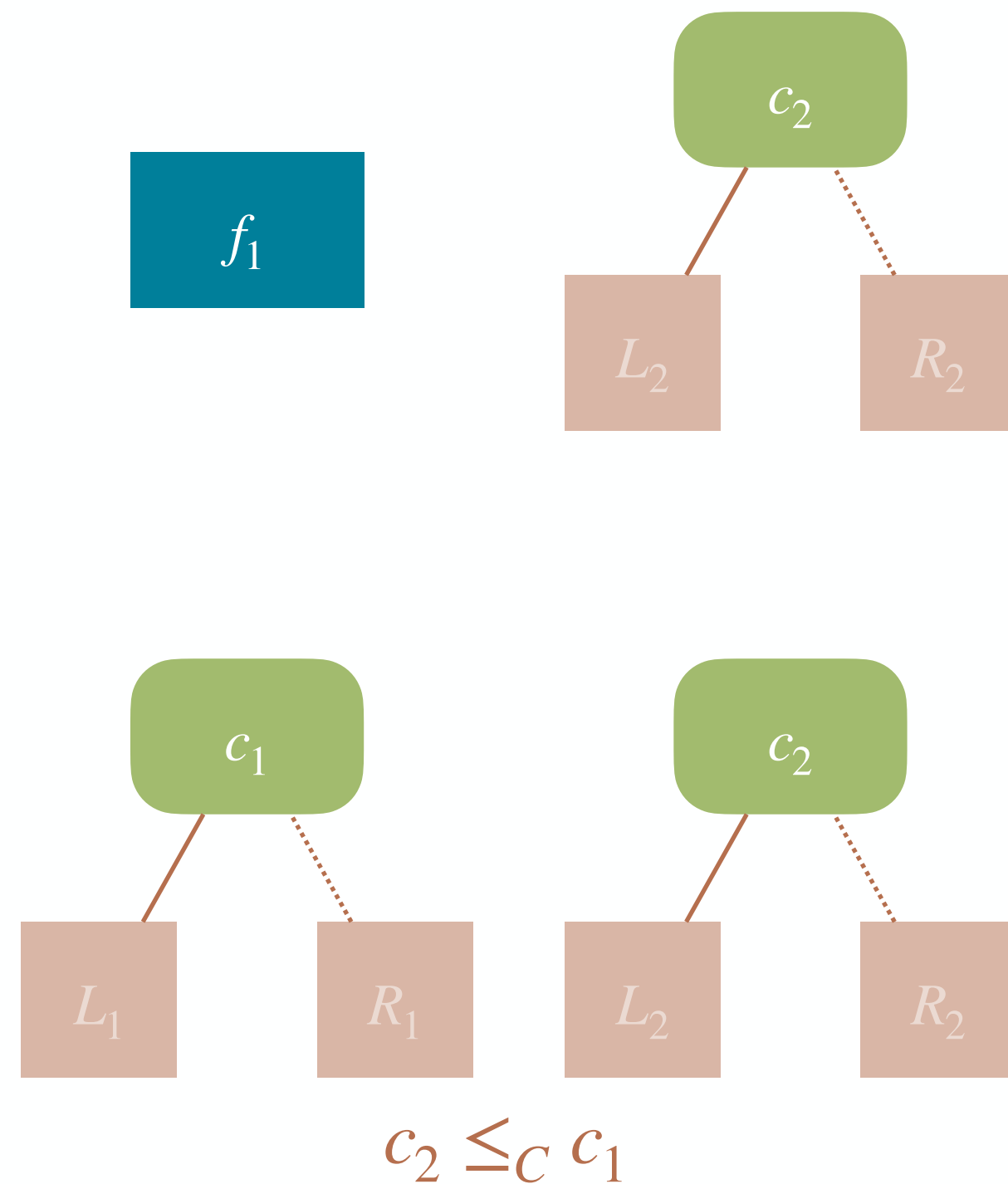


Piecewise-Defined Functions Domain

Widening (continue)

Domain Widening

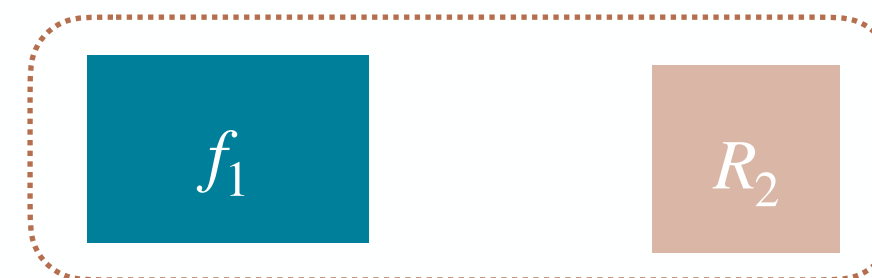
- Case ①



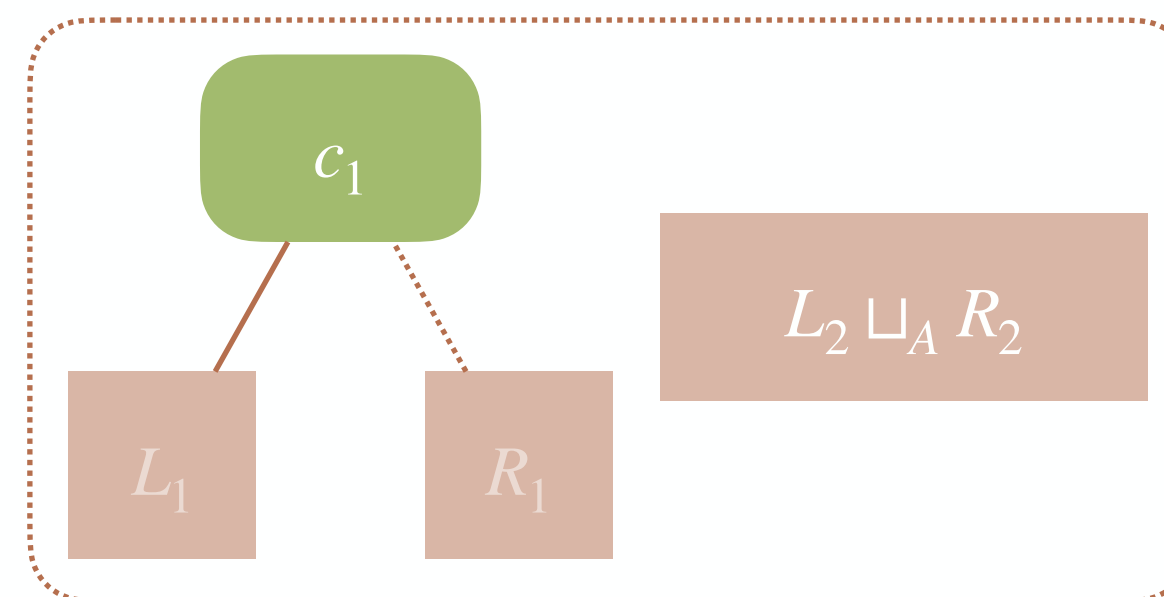
①a) c_2 is redundant



①b) $\neg c_2$ is redundant



①c) c_2 is removed from t_2

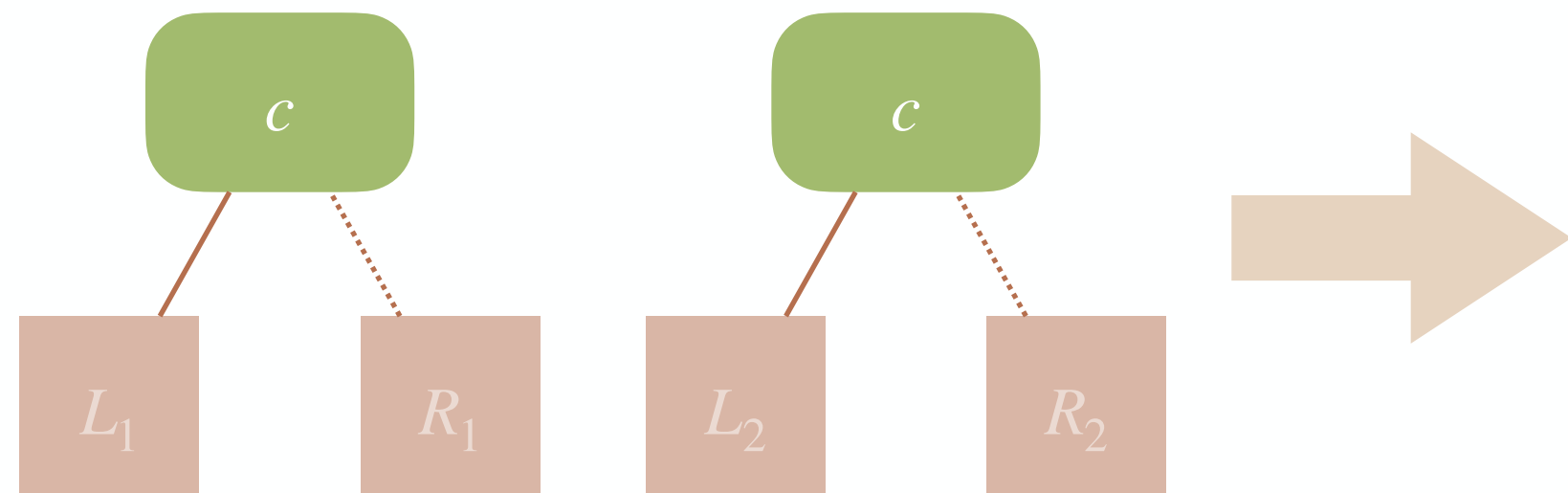


Piecewise-Defined Functions Domain

Widening (continue)

Domain Widening

- Case ② (as for tree unification)
- Case ③



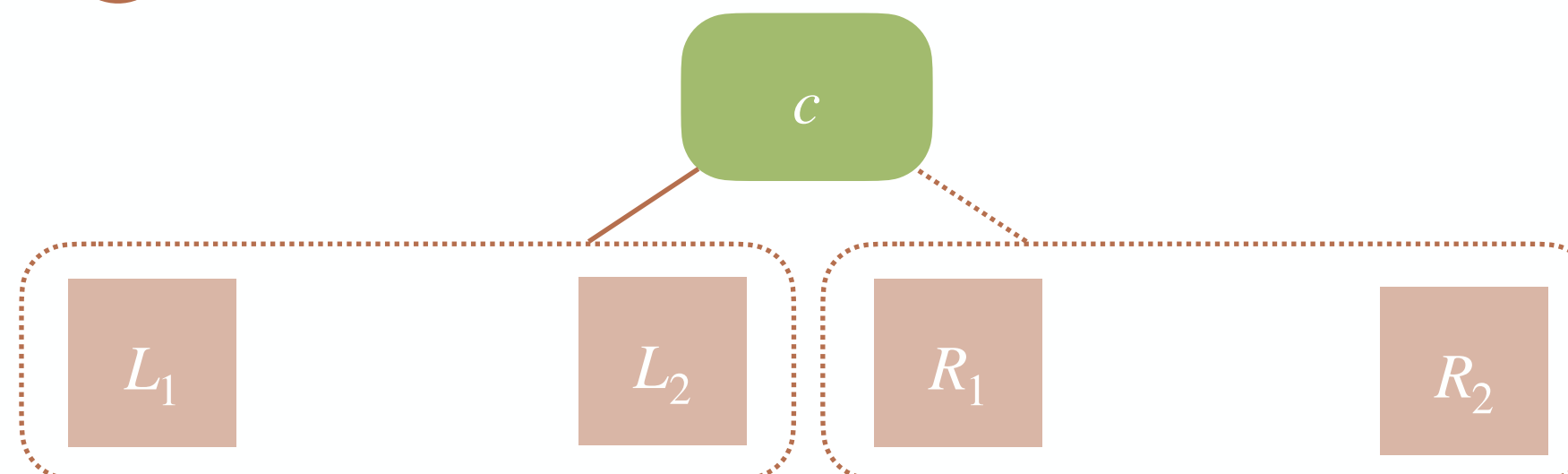
①a) c is redundant



①b) $\neg c$ is redundant



①c) c is kept in t_1 and t_2



Check for Case B or C

Piecewise-Defined Functions Domain

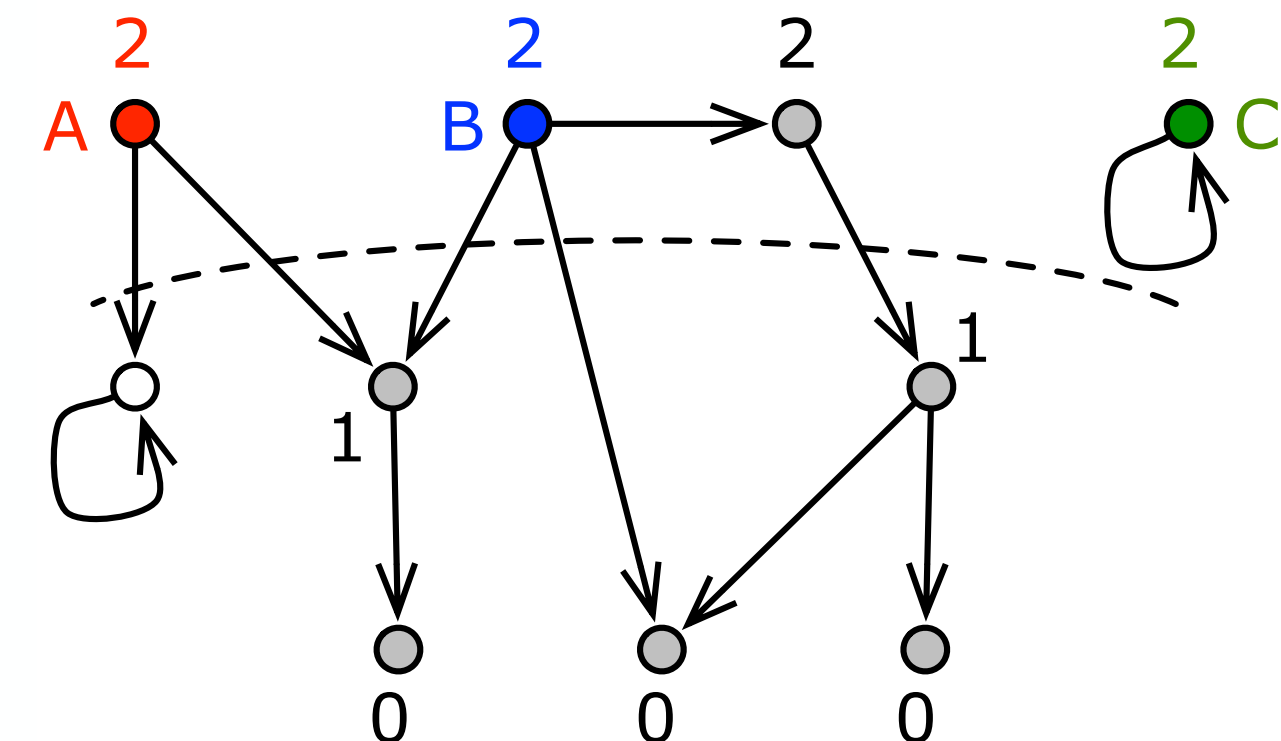
Widening (continue)

Check for Case B or C

1. Recursively descend the trees while *accumulating the linear constraints encountered along the paths* into a set of constraints C

2. f_1 f_2 $\Rightarrow$ f_1 $\top_F$

$$f_1 \in \mathcal{F} \setminus \{ \perp_F, \top_F \} \wedge f_2 \not\leq_F [\alpha_c(C)] f_1$$



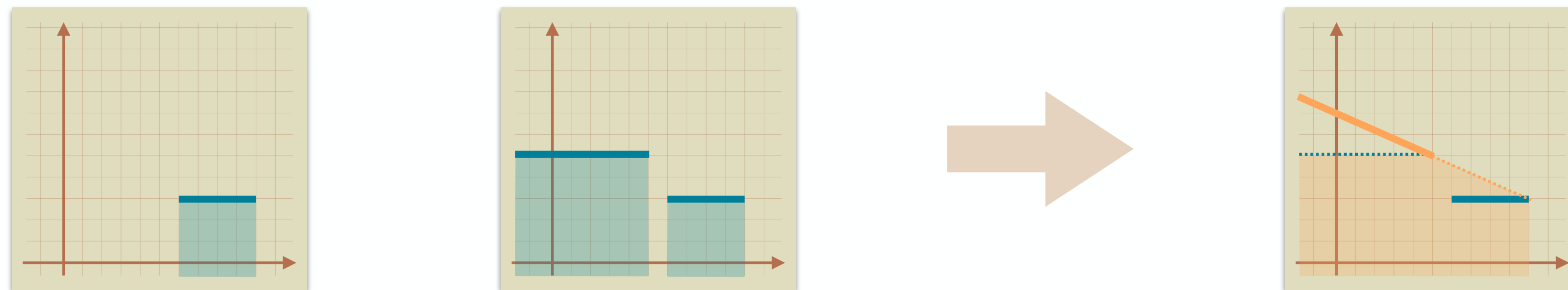
Piecewise-Defined Functions Domain

Widening (continue)

Value Widening

1. Recursively descend the trees while *accumulating the linear constraints encountered along the paths* into a set of constraints C
2. Widen each (defined) leaf node f with respect to each of their adjacent (defined) leaf node $\bar{f}$ using the **extrapolation operator** $\nabla_F [\alpha_C(\bar{C}), \alpha_C(C)]$, where $\bar{C}$ is the set of constraints along the path to $\bar{f}$

Example:



Piecewise-Defined Functions Domain

Abstract Domain Operators

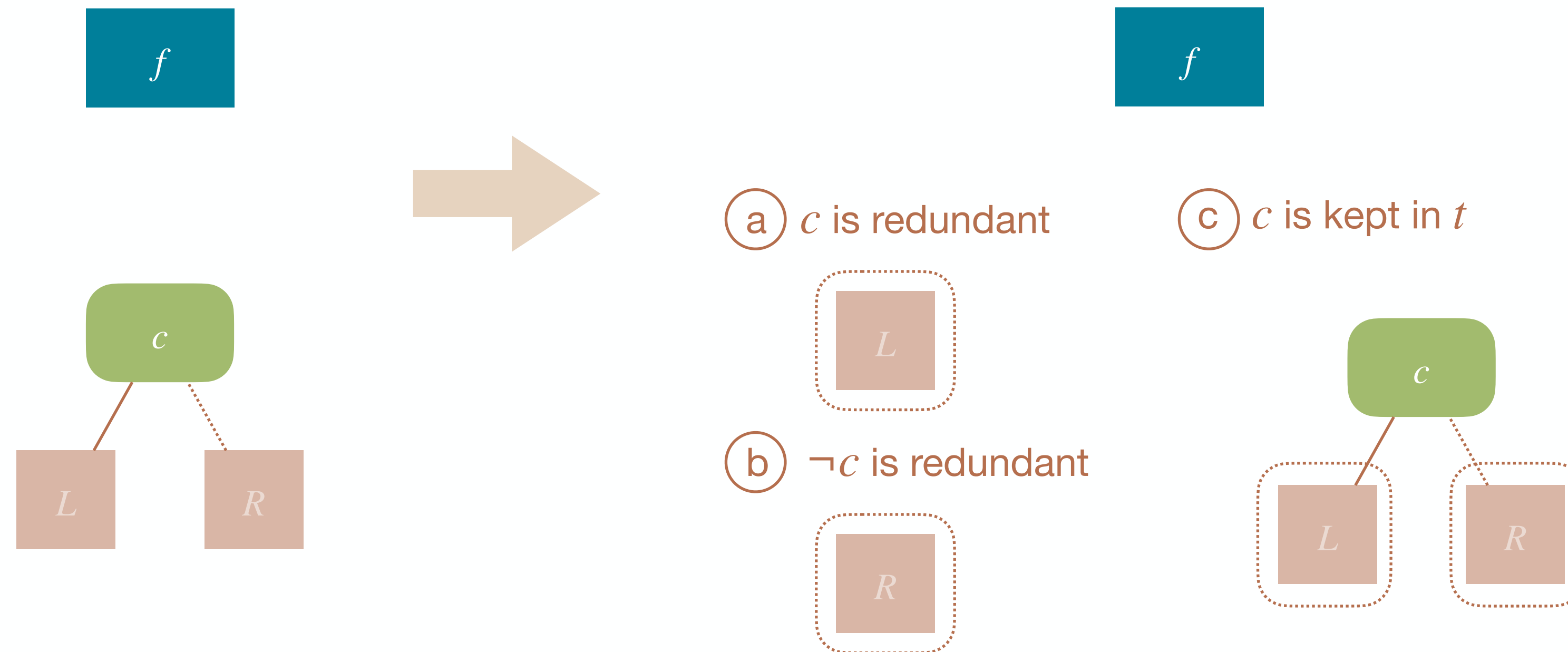
- They manipulate elements in $\mathcal{A}_{\text{NIL}} \stackrel{\text{def}}{=} \{\text{NIL}\} \cup \mathcal{A}$
- The **binary operators** rely on a tree unification algorithm
 - approximation order $\preceq_A$ and computational order $\sqsubseteq_A$
 - approximation join $\vee_A$ and computational join $\sqcup_A$
 - meet $\wedge_A$
 - widening ∇_A
- The **unary operators** rely on a tree pruning algorithm
 - assignment $\overleftarrow{\text{ASSIGN}}_A[[X \leftarrow e]]$
 - test $\text{FILTER}_A[[e]]$

Piecewise-Defined Functions Domain

Tree Pruning

Goal: **add** a set J of **linear constraints** to the decision tree

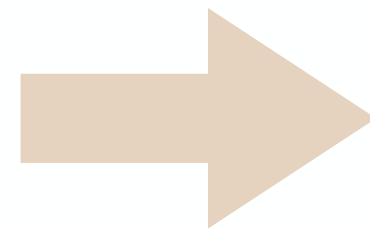
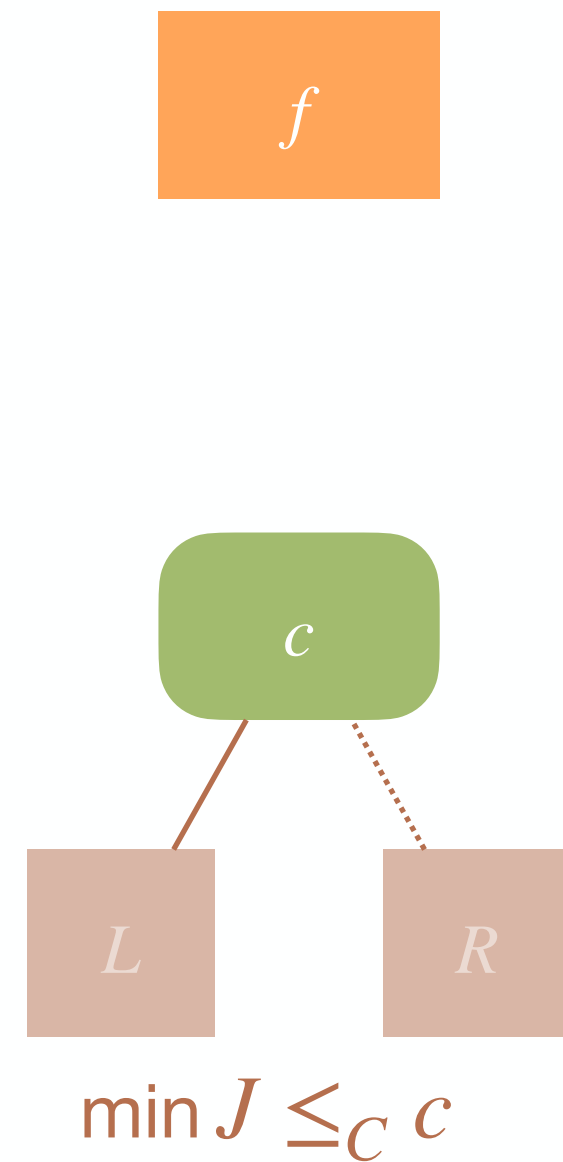
- Base case ($J = \emptyset$)



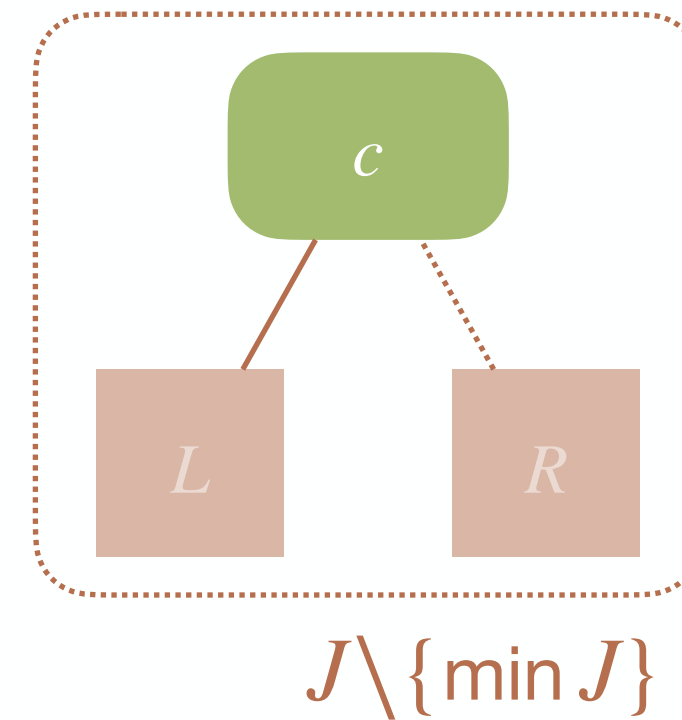
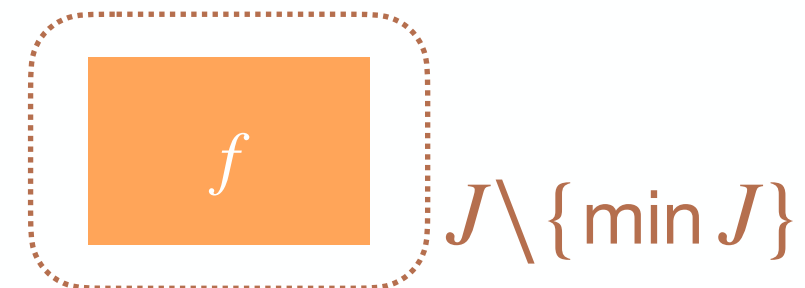
Piecewise-Defined Functions Domain

Tree Pruning (continue)

- Case ①



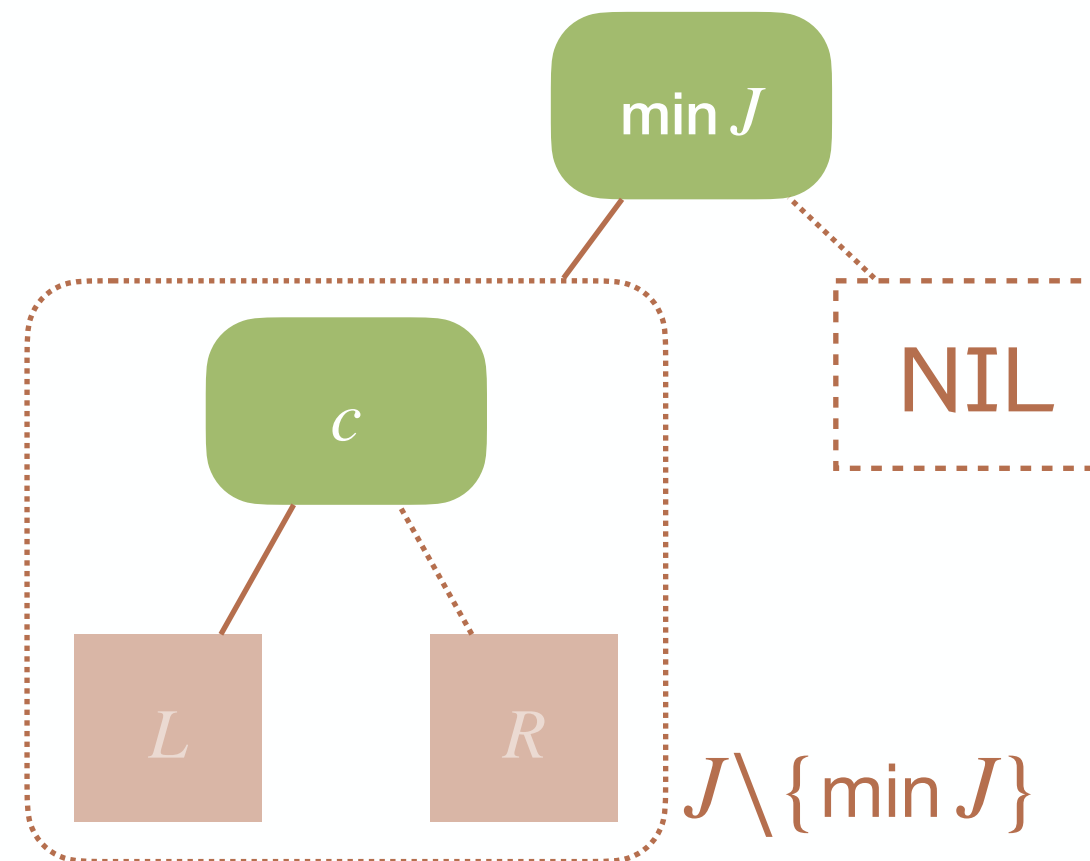
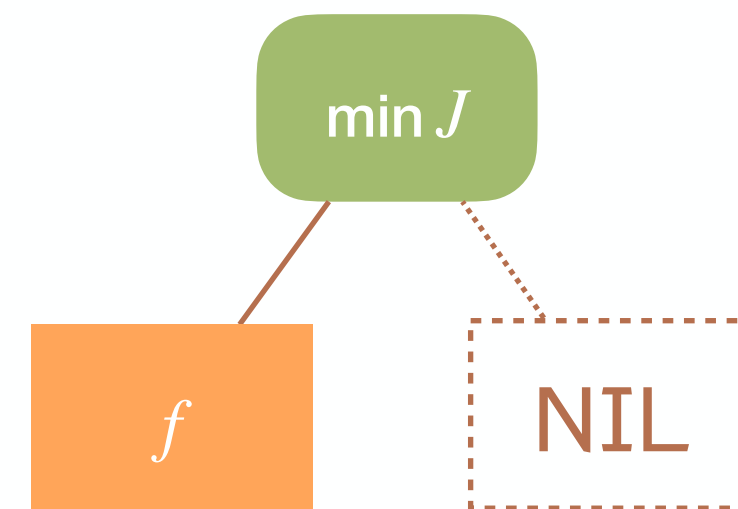
①a) $\min J$ is redundant



①b) $\neg \min J$ is redundant



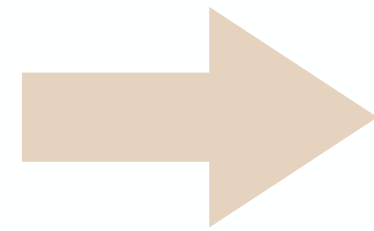
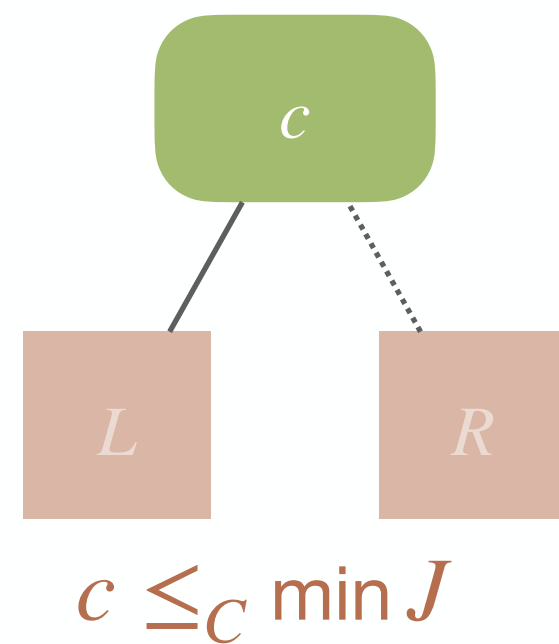
①c) $\min J$ is added to t



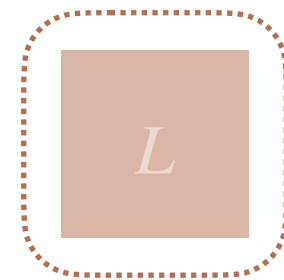
Piecewise-Defined Functions Domain

Tree Pruning (continue)

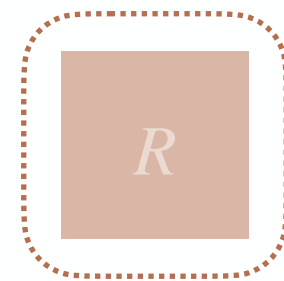
- Case ②



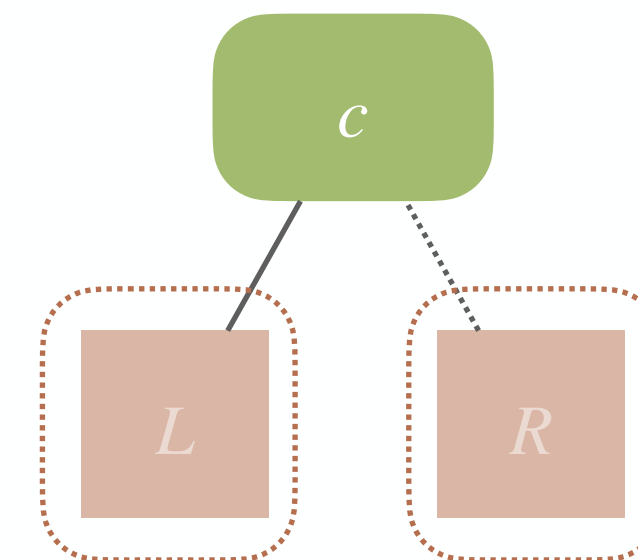
②a) c is redundant



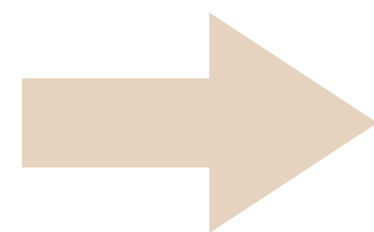
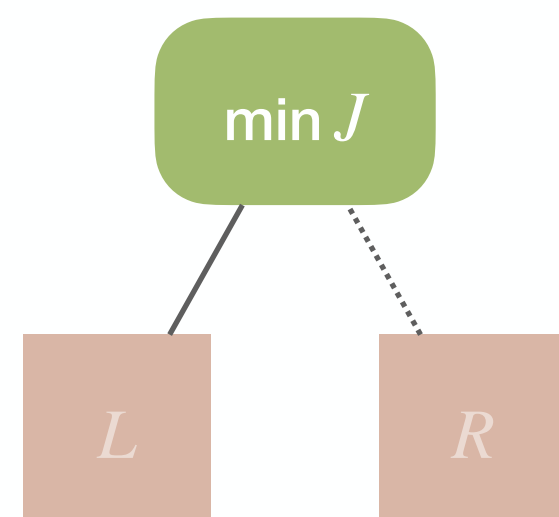
②b) $\neg c$ is redundant



②c) c is kept in t



- Case ③



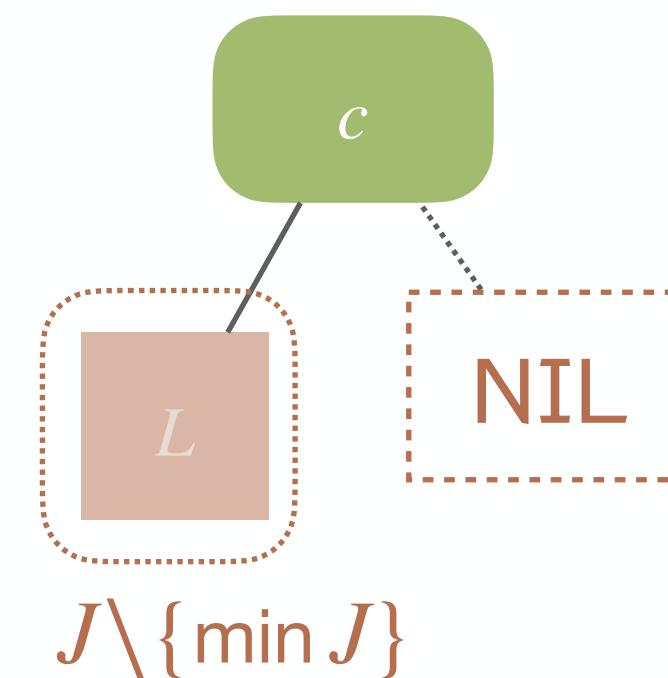
③a) $\min J$ is redundant



③b) $\neg \min J$ is redundant



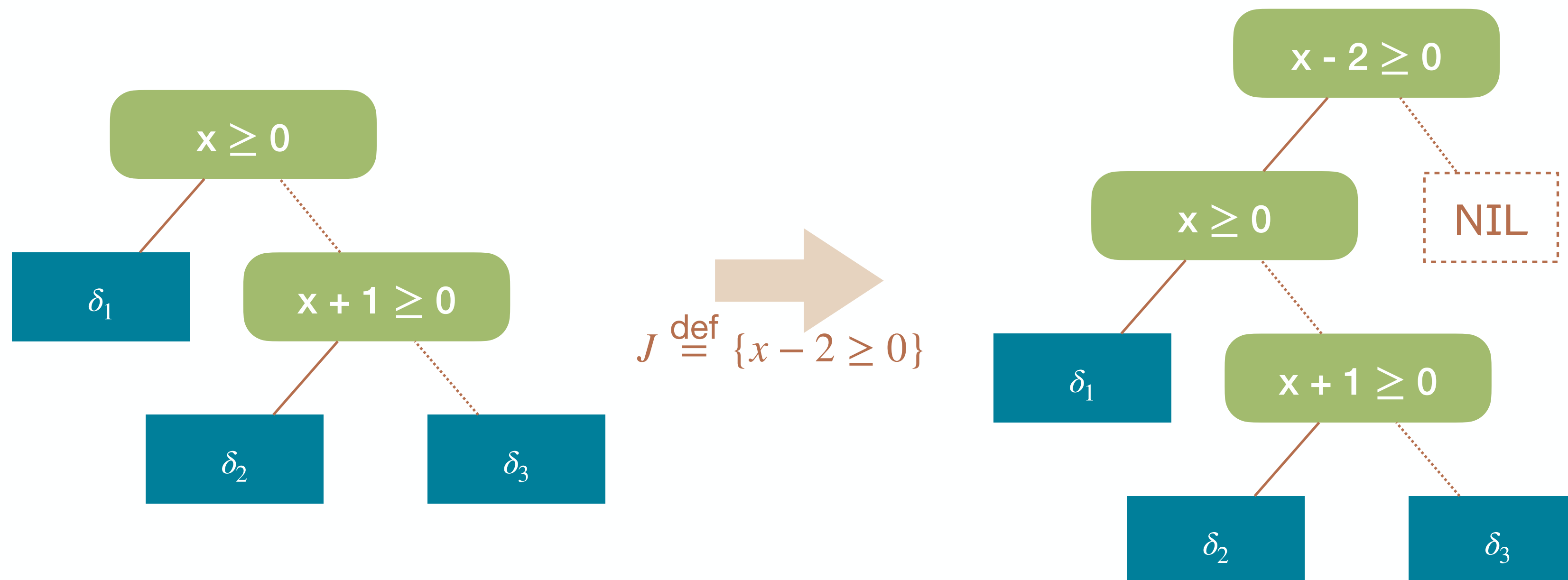
③c) $\min J$ is kept in t



Piecewise-Defined Functions Domain

Tree Pruning (continue)

Example



Piecewise-Defined Functions Domain

Abstract Domain Operators

- They manipulate elements in $\mathcal{A}_{\text{NIL}} \stackrel{\text{def}}{=} \{\text{NIL}\} \cup \mathcal{A}$
- The **binary operators** rely on a tree unification algorithm
 - approximation order $\preceq_A$ and computational order $\sqsubseteq_A$
 - approximation join $\vee_A$ and computational join $\sqcup_A$
 - meet $\wedge_A$
 - widening ∇_A
- The **unary operators** rely on a tree pruning algorithm
 - **assignment** $\overleftarrow{\text{ASSIGN}}_A[[X \leftarrow e]]$
 - test $\text{FILTER}_A[[e]]$

Piecewise-Defined Functions Domain

Assignments

$$\overleftarrow{\text{ASSIGN}}_A[[X \leftarrow e]]$$

- Base case (f)

Apply $\overleftarrow{\text{ASSIGN}}_F[[X \leftarrow e]][\alpha_C(C)]$ on the defined leaf nodes

$$\overleftarrow{\text{ASSIGN}}_F[[X \leftarrow e]][D](f) \stackrel{\text{def}}{=} \begin{cases} \bar{f} & \bar{f} \in \mathcal{F} \setminus \{ \perp_F, \top_F \} \\ \top_F & \text{otherwise} \end{cases} \quad f \in \mathcal{F} \setminus \{ \perp_F, \top_F \}$$

where $\bar{f}(\dots, X_i, X, \dots) \stackrel{\text{def}}{=} \max\{f(\dots, \rho(X_i), v, \dots) + 1 \mid \rho \in \gamma_D(\overleftarrow{\text{ASSIGN}}_D[[X \leftarrow e]]D) \wedge v \in E[[e]]\rho\}$

Example:

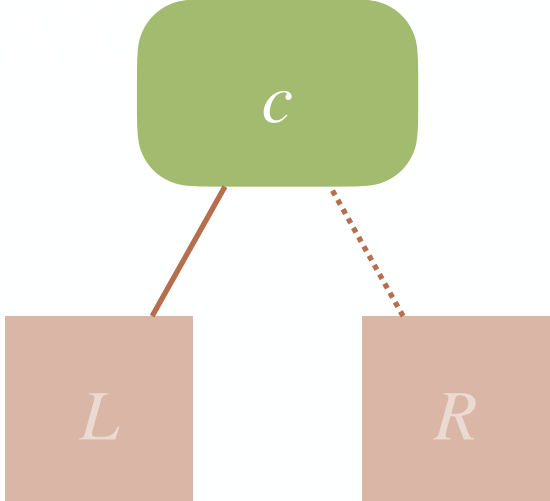
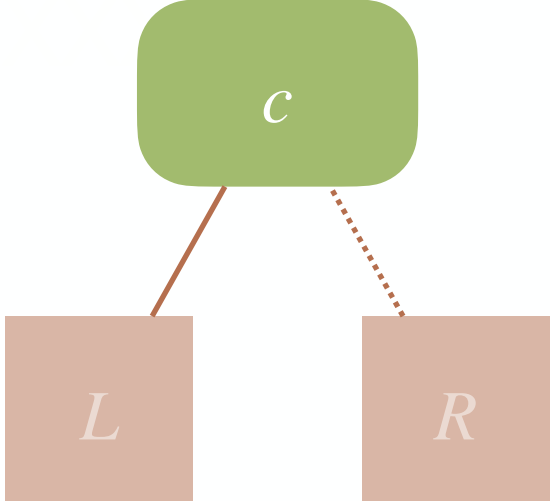
$$\overleftarrow{\text{ASSIGN}}_F[[x \leftarrow x + [1,2]]][\top_D](\lambda x. x + 1) = \lambda x. x + 4$$

(since $f(x + [1,2]) + 1 = x + [1,2] + 1 + 1 = x + [3,4]$ and $\max(3,4) = 4$)

Piecewise-Defined Functions Domain

Assignments

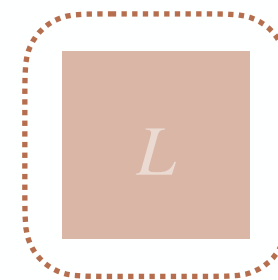
$$\overleftarrow{\text{ASSIGN}}_A[[X \leftarrow e]]$$

-  c  L R
Convert $\overleftarrow{\text{ASSIGN}}_D[[X \leftarrow e]](\alpha_C(\{c\}))$ and $\overleftarrow{\text{ASSIGN}}_D[[X \leftarrow e]](\alpha_C(\{\neg c\}))$
into sets I and J of linear constraints *in canonical form*

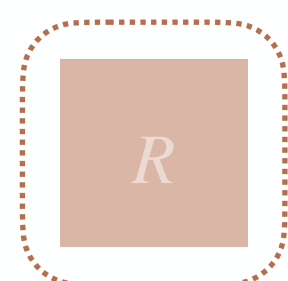
case ① $I = J = \emptyset$



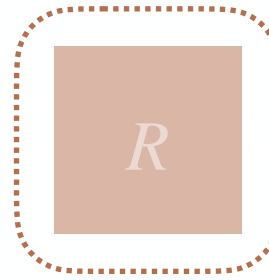
case ② $I = \emptyset \wedge \perp_C \in J$



case ③ $\perp_C \in I \wedge J = \emptyset$



case ④

1. perform **tree pruning** on  L and  R
2. join the results with $\vee_A$

Piecewise-Defined Functions Domain

Assignments

function $f(x)$ {

1 $a \leftarrow [-\infty, +\infty]$

2 $z \leftarrow 10$

3 if $(a*a \geq 0)$ then

while 4 $(z \geq 0)$ do

5 $z \leftarrow z - x$

done 6

else

while 7 $(z \geq x)$ do

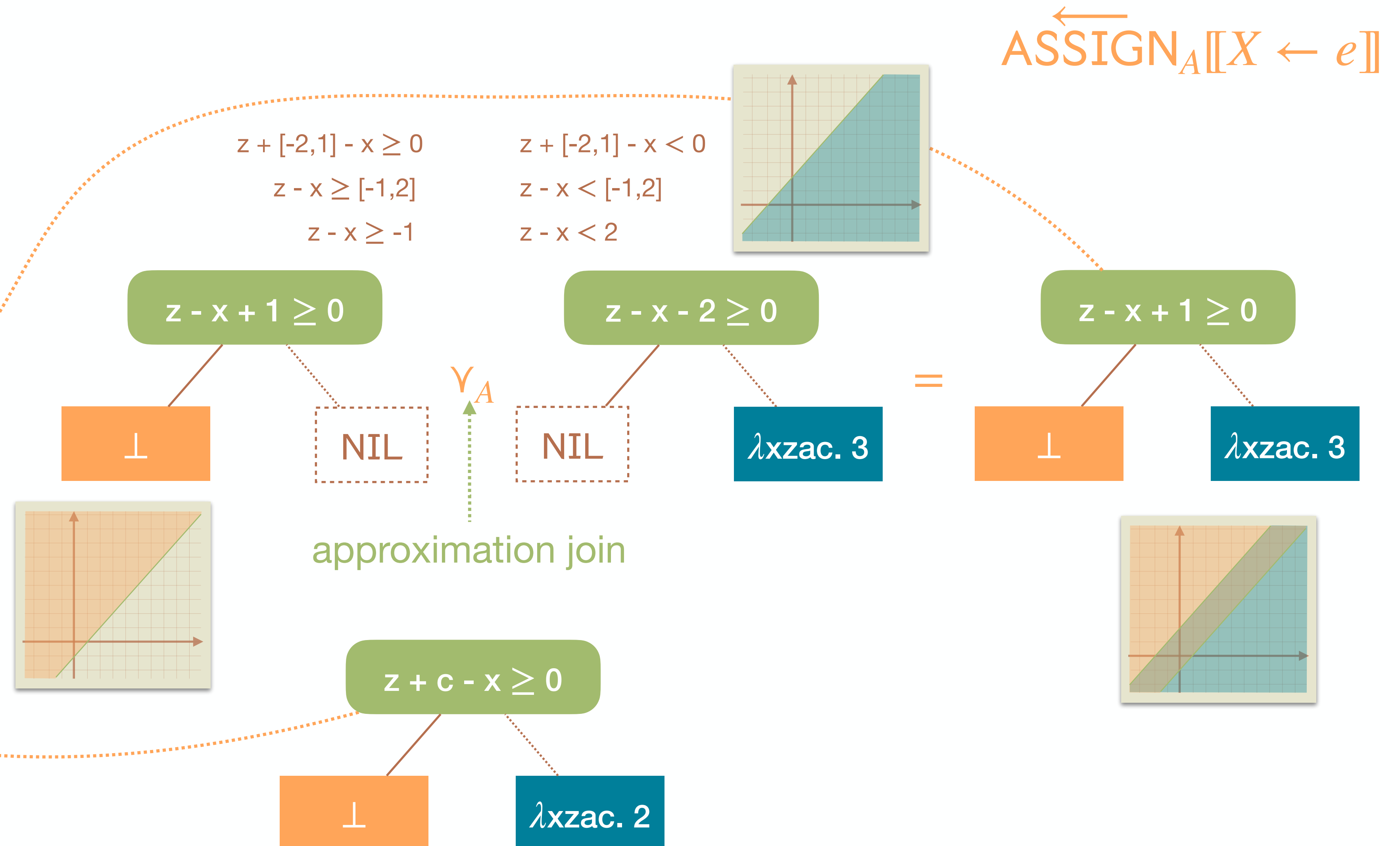
8 $c \leftarrow [-2, 1]$

9 $z \leftarrow z + c$

done 10

end

} 11



Piecewise-Defined Functions Domain

Abstract Domain Operators

- They manipulate elements in $\mathcal{A}_{\text{NIL}} \stackrel{\text{def}}{=} \{\text{NIL}\} \cup \mathcal{A}$
- The **binary operators** rely on a tree unification algorithm
 - approximation order $\preceq_A$ and computational order $\sqsubseteq_A$
 - approximation join $\vee_A$ and computational join $\sqcup_A$
 - meet $\wedge_A$
 - widening ∇_A
- The **unary operators** rely on a tree pruning algorithm
 - assignment $\overleftarrow{\text{ASSIGN}}_A[[X \leftarrow e]]$
 - **test** $\text{FILTER}_A[[e]]$

Piecewise-Defined Functions Domain

Tests

$\text{FILTER}_A[[e]]$

1. Recursively descend the tree and apply STEP_F on the defined leaf nodes to account for one more execution step needed before termination:

$$\text{STEP}_F(f) \stackrel{\text{def}}{=} \lambda X_1, \dots, X_k. f(X_1, \dots, X_k) + 1 \qquad f \in \mathcal{F} \setminus \{ \perp_F, \top_F \}$$

2. **Convert e into a set J of linear constraints in canonical form**

Example: $\alpha_C(\text{FILTER}_D[[e]] \top_D)$

where $\langle \mathcal{D}, \sqsubseteq_D \rangle$ is the underlying numerical domain

3. Perform **tree pruning** with J

Abstract Definite Termination Semantics

For each program instruction stmt , we define a transformer $\mathcal{R}_M^\#[\text{stmt}] : \mathcal{A} \rightarrow \mathcal{A}$:

- $\mathcal{R}_M^\#[\![X \leftarrow e]\!]t \stackrel{\text{def}}{=} \overleftarrow{\text{ASSIGN}}_A[\![X \leftarrow e]\!]t$

Lemma (Soundness)

$$\mathcal{R}_M[\![X \leftarrow e]\!]\gamma_A(t) \preceq \gamma_A(\mathcal{R}_M^\#[\![X \leftarrow e]\!]t)$$

Abstract Definite Termination Semantics

For each program instruction stmt , we define a transformer $\mathcal{R}_M^\#[\text{stmt}] : \mathcal{A} \rightarrow \mathcal{A}$:

- $\mathcal{R}_M^\#[\![X \leftarrow e]\!]t \stackrel{\text{def}}{=} \overleftarrow{\text{ASSIGN}}_A[\![X \leftarrow e]\!]t$
- $\mathcal{R}_M^\#[\![\text{if } e \bowtie 0 \text{ then } s \text{ end}]\!]t \stackrel{\text{def}}{=} \text{FILTER}_A[\![e \bowtie 0]\!](\mathcal{R}_M^\#[\![s_1]\!]t) \vee_T \text{FILTER}_A[\![e \bowtie 0]\!](t)$

Lemma (Soundness)

$$\mathcal{R}_M[\![\text{if } e \bowtie 0 \text{ then } s \text{ end}]\!]\gamma_A(t) \preceq \gamma_A(\mathcal{R}_M^\#[\![\text{if } e \bowtie 0 \text{ then } s \text{ end}]\!]t)$$

Abstract Definite Termination Semantics

For each program instruction stmt , we define a transformer $\mathcal{R}_M^\#[\text{stmt}] : \mathcal{A} \rightarrow \mathcal{A}$:

- $\mathcal{R}_M^\#[\![X \leftarrow e]\!]t \stackrel{\text{def}}{=} \overleftarrow{\text{ASSIGN}}_A[\![X \leftarrow e]\!]t$
- $\mathcal{R}_M^\#[\![\text{if } e \bowtie 0 \text{ then } s \text{ end}]\!]t \stackrel{\text{def}}{=} \text{FILTER}_A[\![e \bowtie 0]\!](\mathcal{R}_M^\#[\![s_1]\!]t) \vee_T \text{FILTER}_A[\![e \bowtie 0]\!](t)$
- $\mathcal{R}_M^\#[\![\text{while } e \bowtie 0 \text{ do } s \text{ done}]\!]t \stackrel{\text{def}}{=} \text{lfp}^\# \overline{F}_M^\#$

where $\overline{F}_M^\#(x) \stackrel{\text{def}}{=} \text{FILTER}_A[\![e \bowtie 0]\!](\mathcal{R}_M^\#[\![s]\!]x) \vee_T \text{FILTER}_A[\![e \bowtie 0]\!](t)$

Lemma (Soundness)

$$\mathcal{R}_M[\![\text{while } e \bowtie 0 \text{ do } s \text{ done}]\!]\gamma_A(t) \preceq \gamma_A(\mathcal{R}_M^\#[\![\text{while } e \bowtie 0 \text{ do } s \text{ done}]\!]t)$$

Abstract Definite Termination Semantics

For each program instruction stmt , we define a transformer $\mathcal{R}_M^\#[\text{stmt}] : \mathcal{A} \rightarrow \mathcal{A}$:

- $\mathcal{R}_M^\#[\![X \leftarrow e]\!]t \stackrel{\text{def}}{=} \overleftarrow{\text{ASSIGN}}_A[\![X \leftarrow e]\!]t$
- $\mathcal{R}_M^\#[\![\text{if } e \bowtie 0 \text{ then } s \text{ end}]\!]t \stackrel{\text{def}}{=} \text{FILTER}_A[\![e \bowtie 0]\!](\mathcal{R}_M^\#[\![s_1]\!]t) \vee_T \text{FILTER}_A[\![e \bowtie 0]\!](t)$
- $\mathcal{R}_M^\#[\![\text{while } e \bowtie 0 \text{ do } s \text{ done}]\!]t \stackrel{\text{def}}{=} \text{lfp}^\# \overline{F}_M^\#$

where $\overline{F}_M^\#(x) \stackrel{\text{def}}{=} \text{FILTER}_A[\![e \bowtie 0]\!](\mathcal{R}_M^\#[\![s]\!]x) \vee_T \text{FILTER}_A[\![e \bowtie 0]\!](t)$

- $\mathcal{R}_M^\#[\![s_1; s_2]\!]t \stackrel{\text{def}}{=} \mathcal{R}_M^\#[\![s_1]\!](\mathcal{R}_M^\#[\![s_2]\!]t)$

Lemma (Soundness)

$$\mathcal{R}_M[\![s_1; s_2]\!]\gamma_A(t) \preceq \gamma_A(\mathcal{R}_M^\#[\![s_1; s_2]\!]t)$$

Abstract Definite Termination Semantics

Definition

The **abstract definite termination semantics** $\mathcal{R}_M^\# \llbracket s^\ell \rrbracket \in \mathcal{A}$ of a program s^ℓ is:

$$\mathcal{R}_M^\# \llbracket s^\ell \rrbracket \stackrel{\text{def}}{=} \mathcal{R}_M^\# \llbracket s \rrbracket (\text{LEAF}: \lambda X_1, \dots, X_k. 0)$$

where $\mathcal{R}_M^\# \llbracket s \rrbracket: \mathcal{A} \rightarrow \mathcal{A}$ is the abstract definite termination semantics of each program instruction s

Theorem (Soundness)

$$\mathcal{R}_M \llbracket s^\ell \rrbracket \preceq \gamma_A(\mathcal{R}_M^\# \llbracket s^\ell \rrbracket)$$

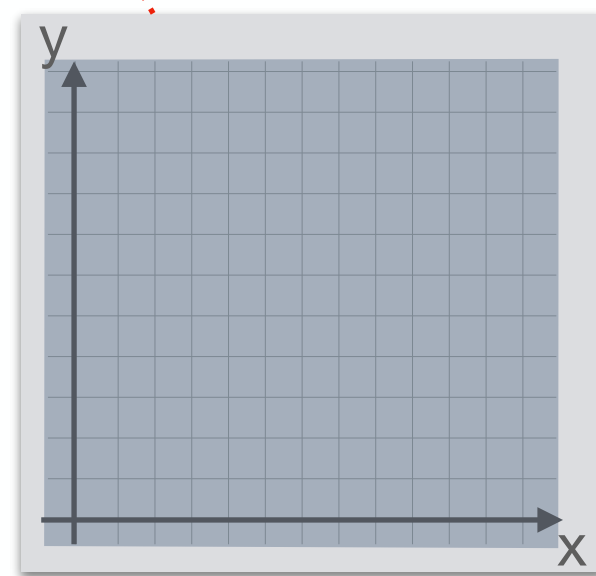
Corollary (Soundness)

A program s^ℓ **must terminate** starting from a set of initial states I if $I \subseteq \text{dom}(\gamma_A(\mathcal{R}_M^\# \llbracket s^\ell \rrbracket))$

Abstract Definite Termination Semantics

Example

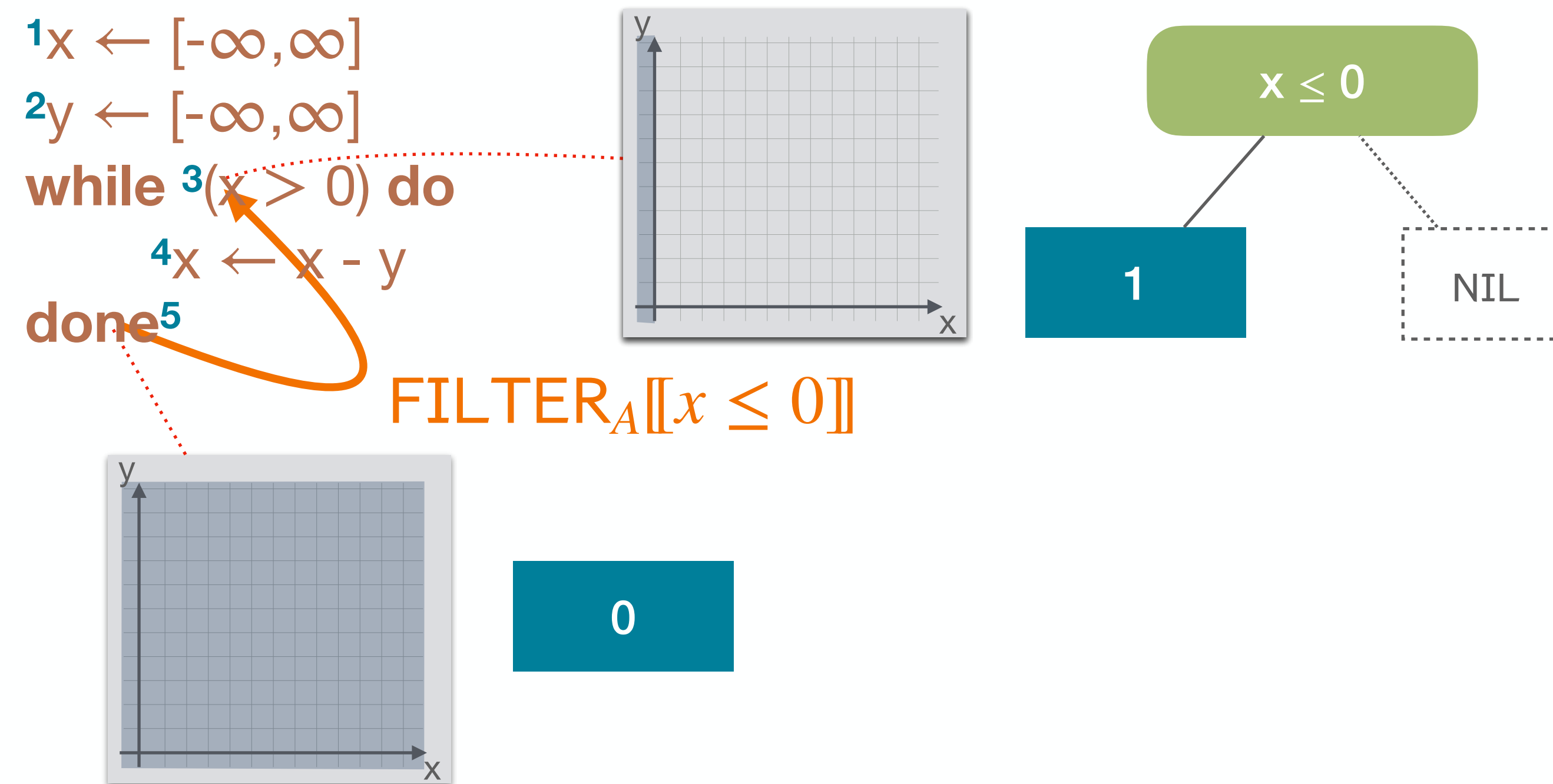
```
1  $x \leftarrow [-\infty, \infty]$   
2  $y \leftarrow [-\infty, \infty]$   
while 3  $(x > 0)$  do  
    4  $x \leftarrow x - y$   
done 5
```



0

Abstract Definite Termination Semantics

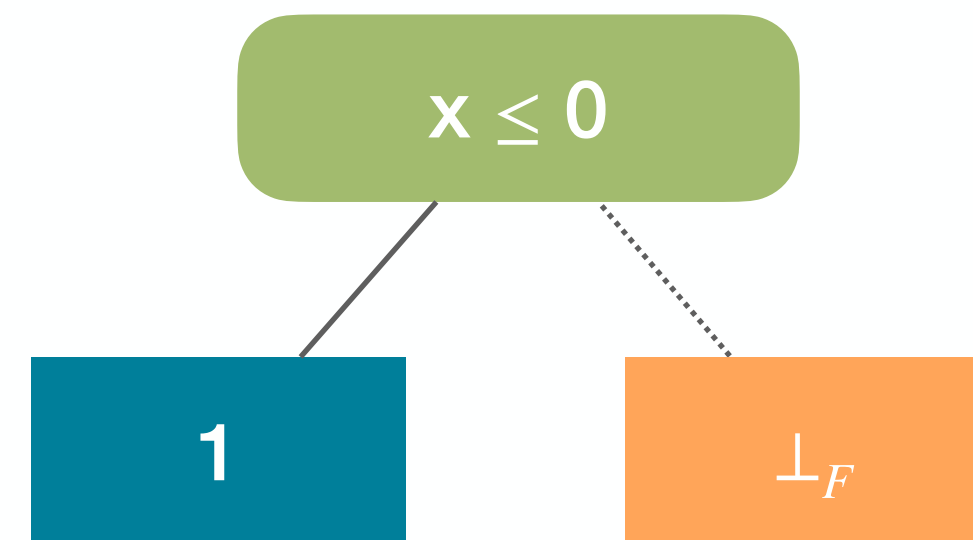
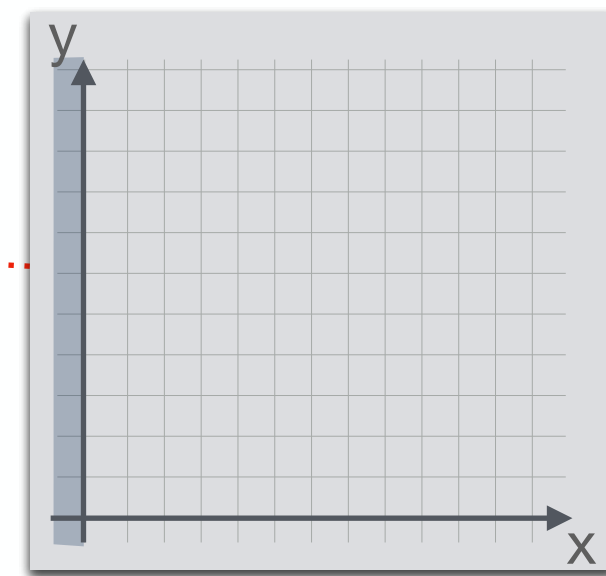
Example



Abstract Definite Termination Semantics

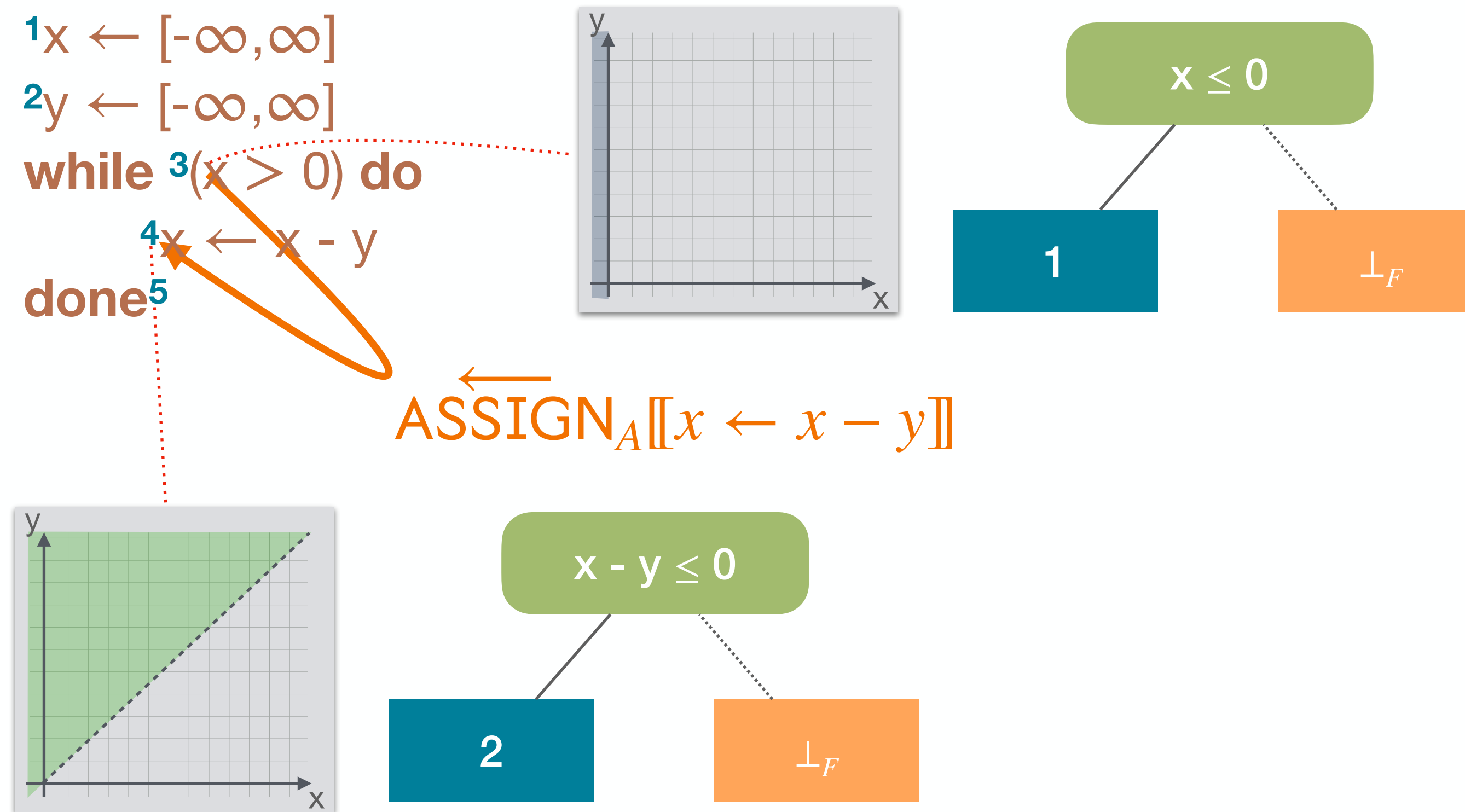
Example

```
1  $x \leftarrow [-\infty, \infty]$   
2  $y \leftarrow [-\infty, \infty]$   
while 3  $(x > 0)$  do  
    4  $x \leftarrow x - y$   
done 5
```



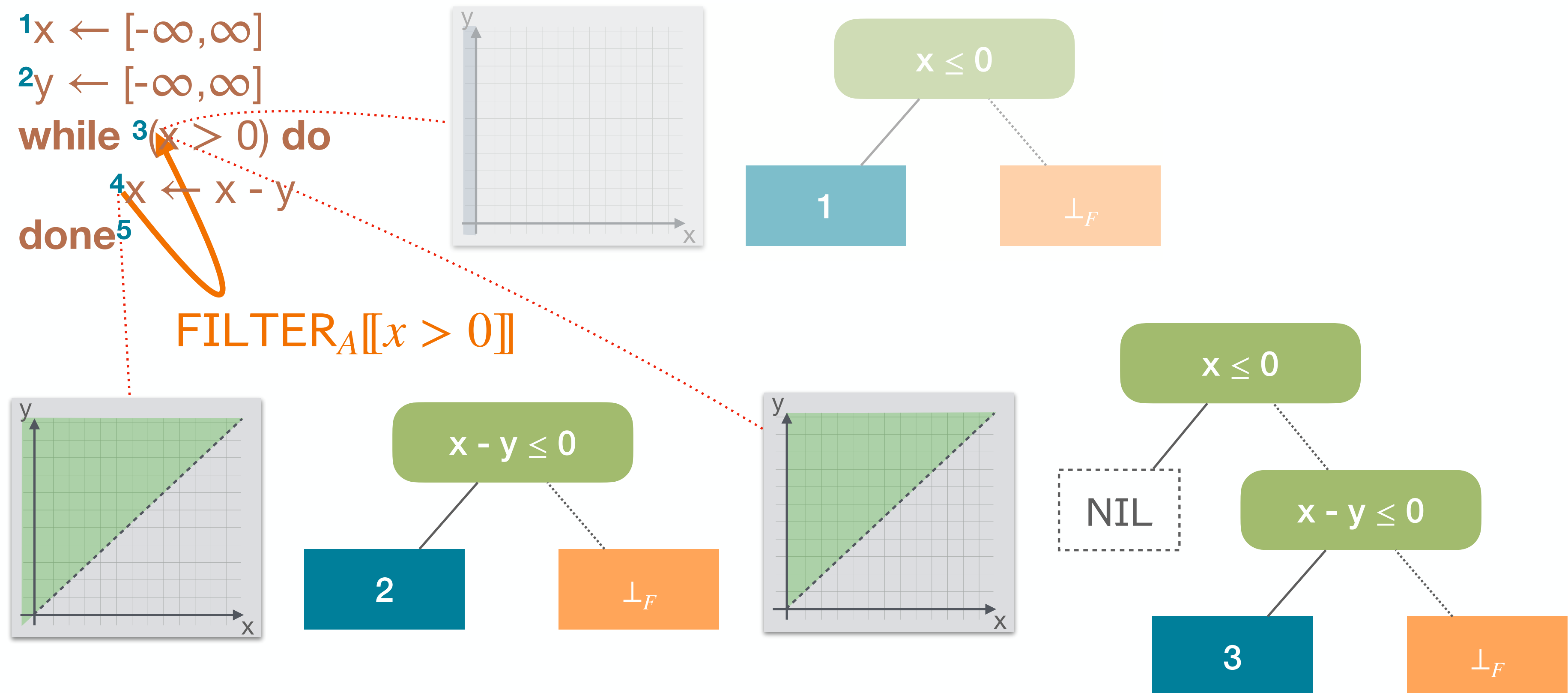
Abstract Definite Termination Semantics

Example



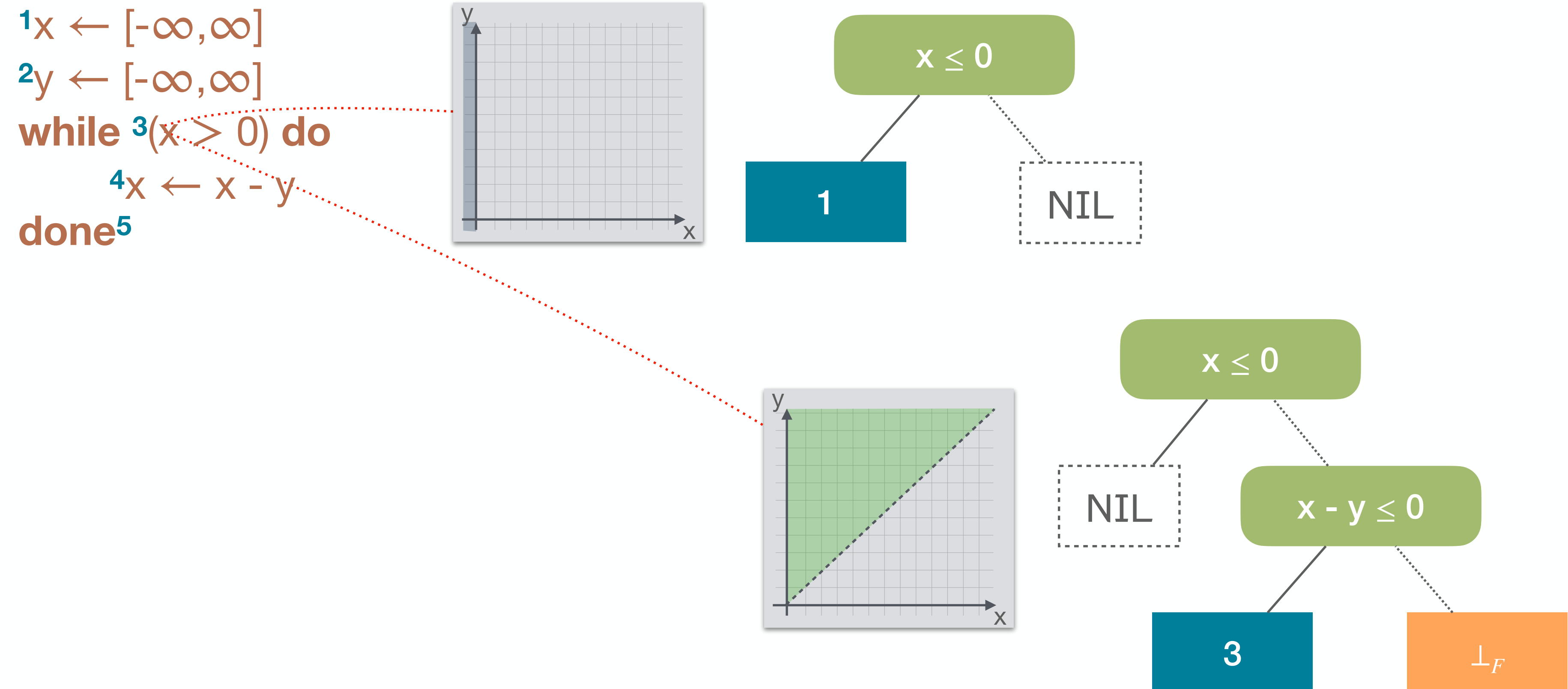
Abstract Definite Termination Semantics

Example



Abstract Definite Termination Semantics

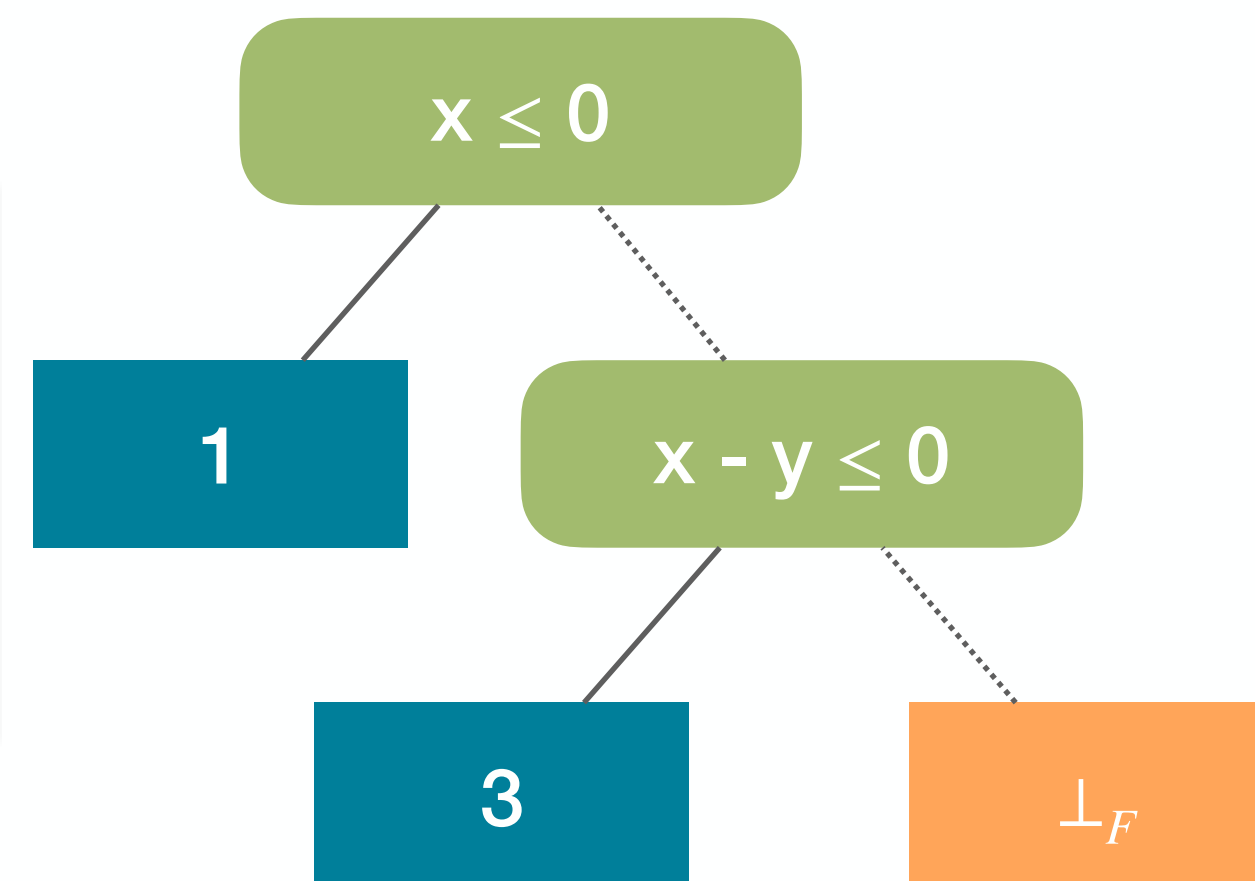
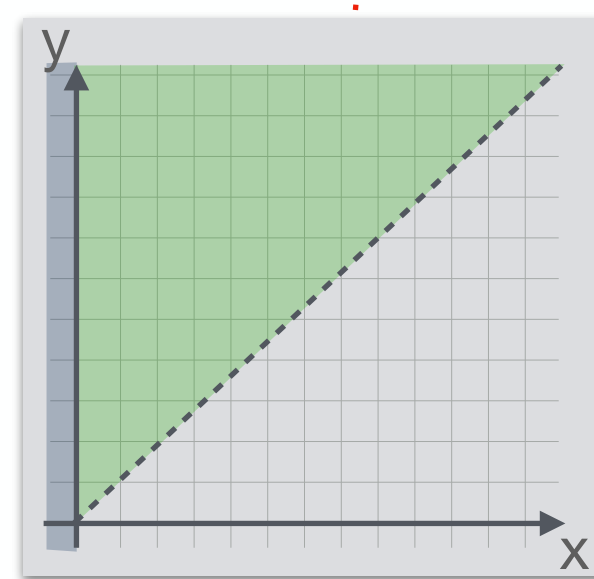
Example



Abstract Definite Termination Semantics

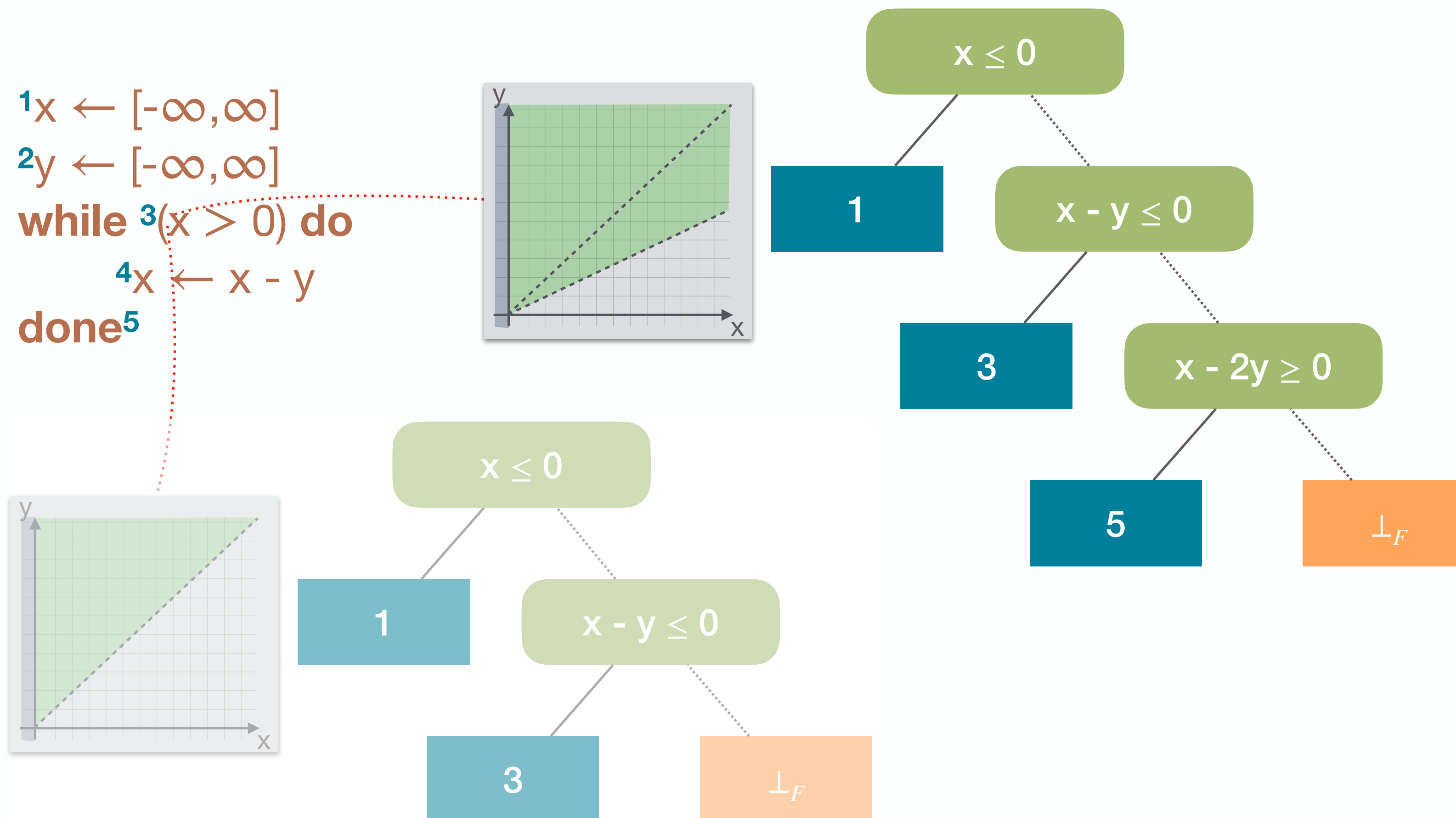
Example

```
1  $x \leftarrow [-\infty, \infty]$   
2  $y \leftarrow [-\infty, \infty]$   
while 3  $(x > 0)$  do  
  4  $x \leftarrow x - y$   
done 5
```



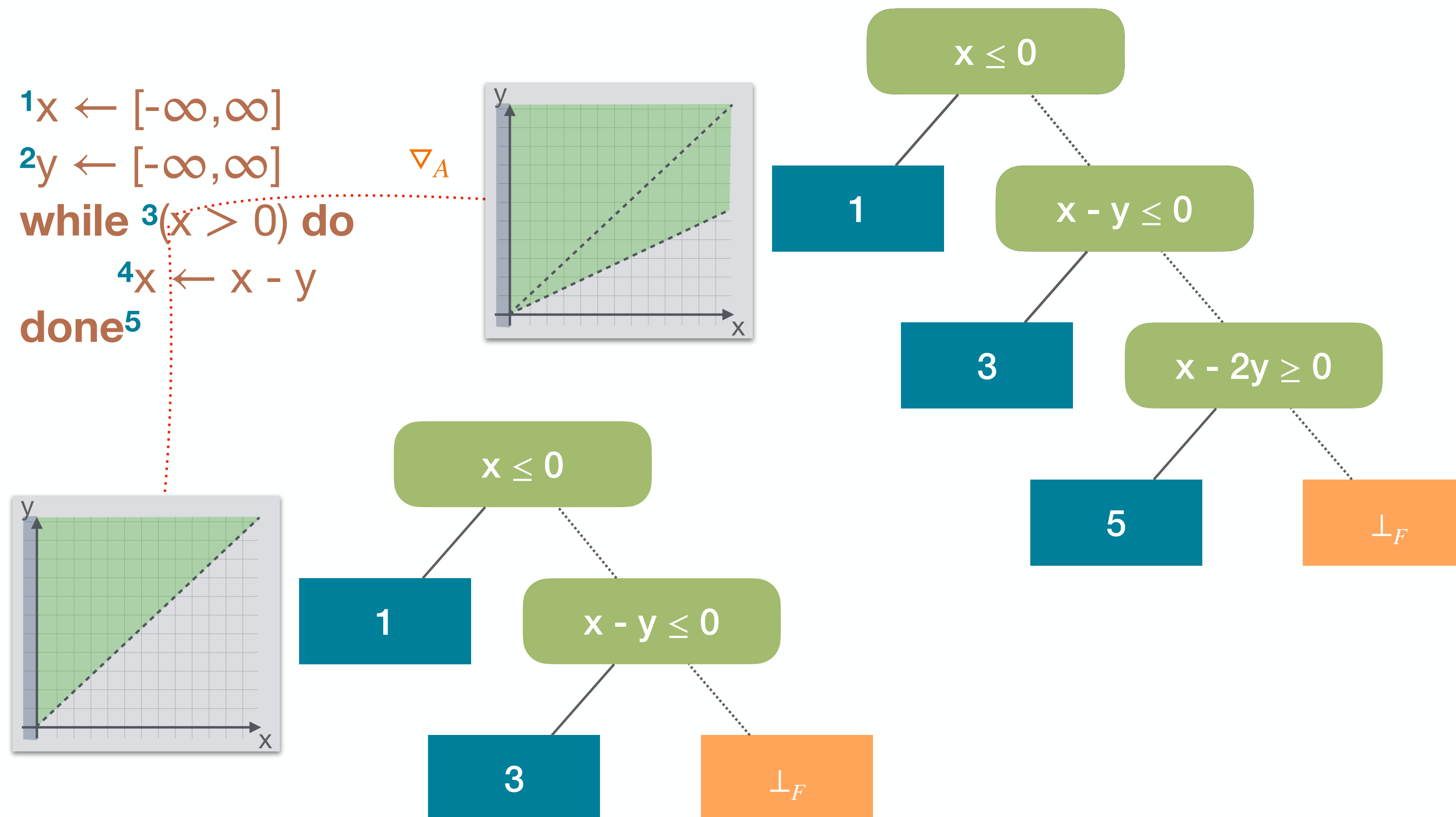
Abstract Definite Termination Semantics

Example



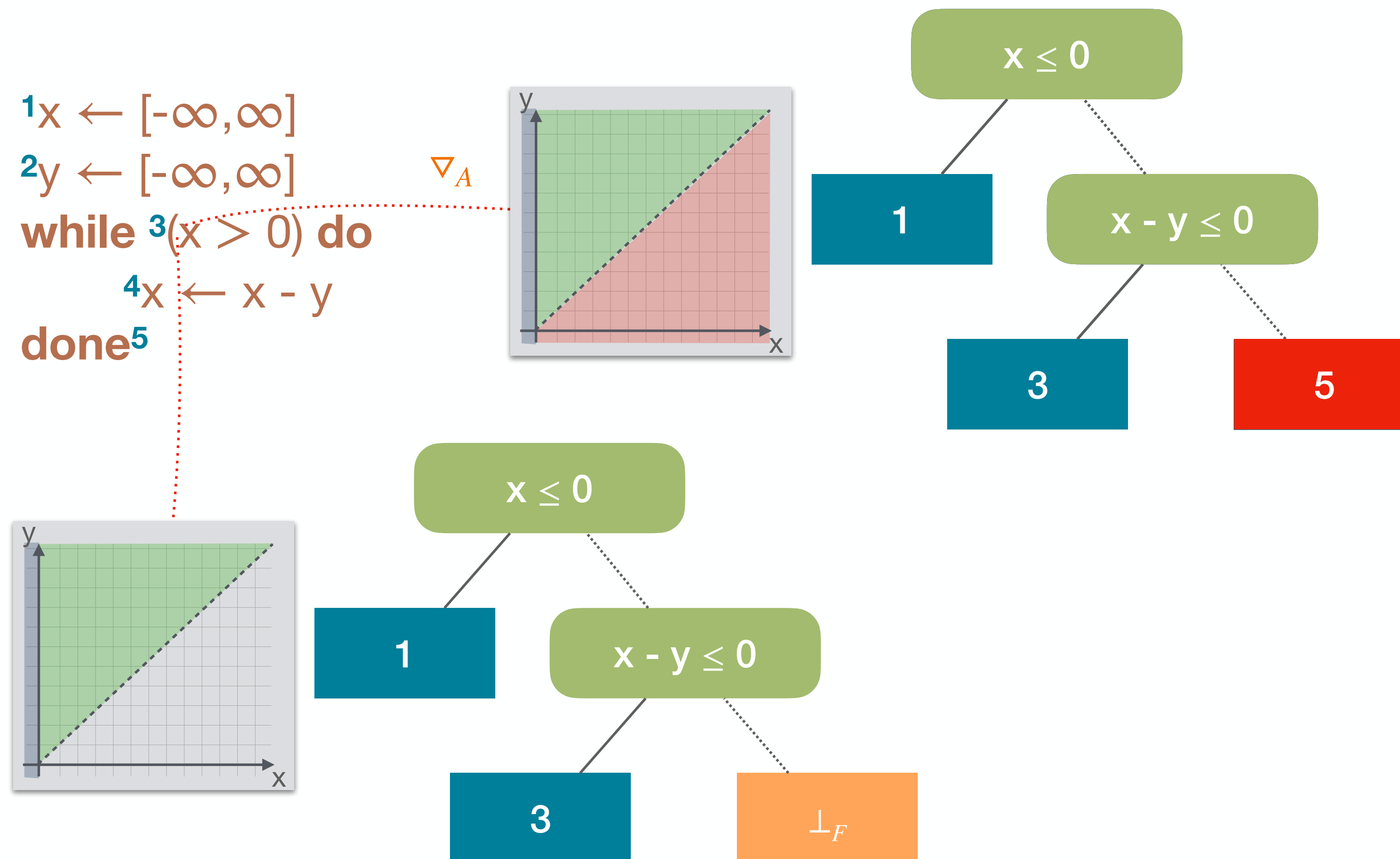
Abstract Definite Termination Semantics

Example



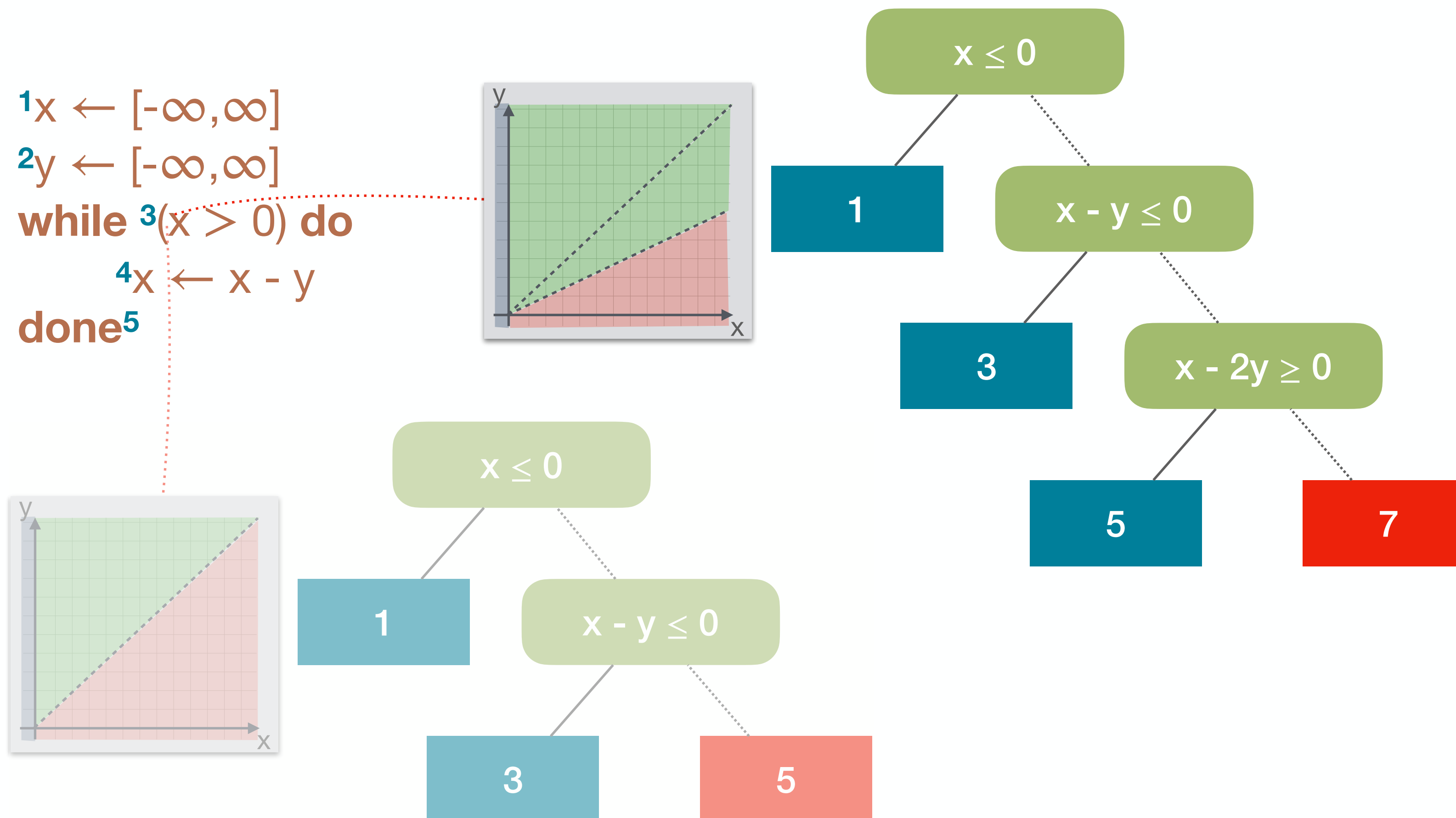
Abstract Definite Termination Semantics

Example



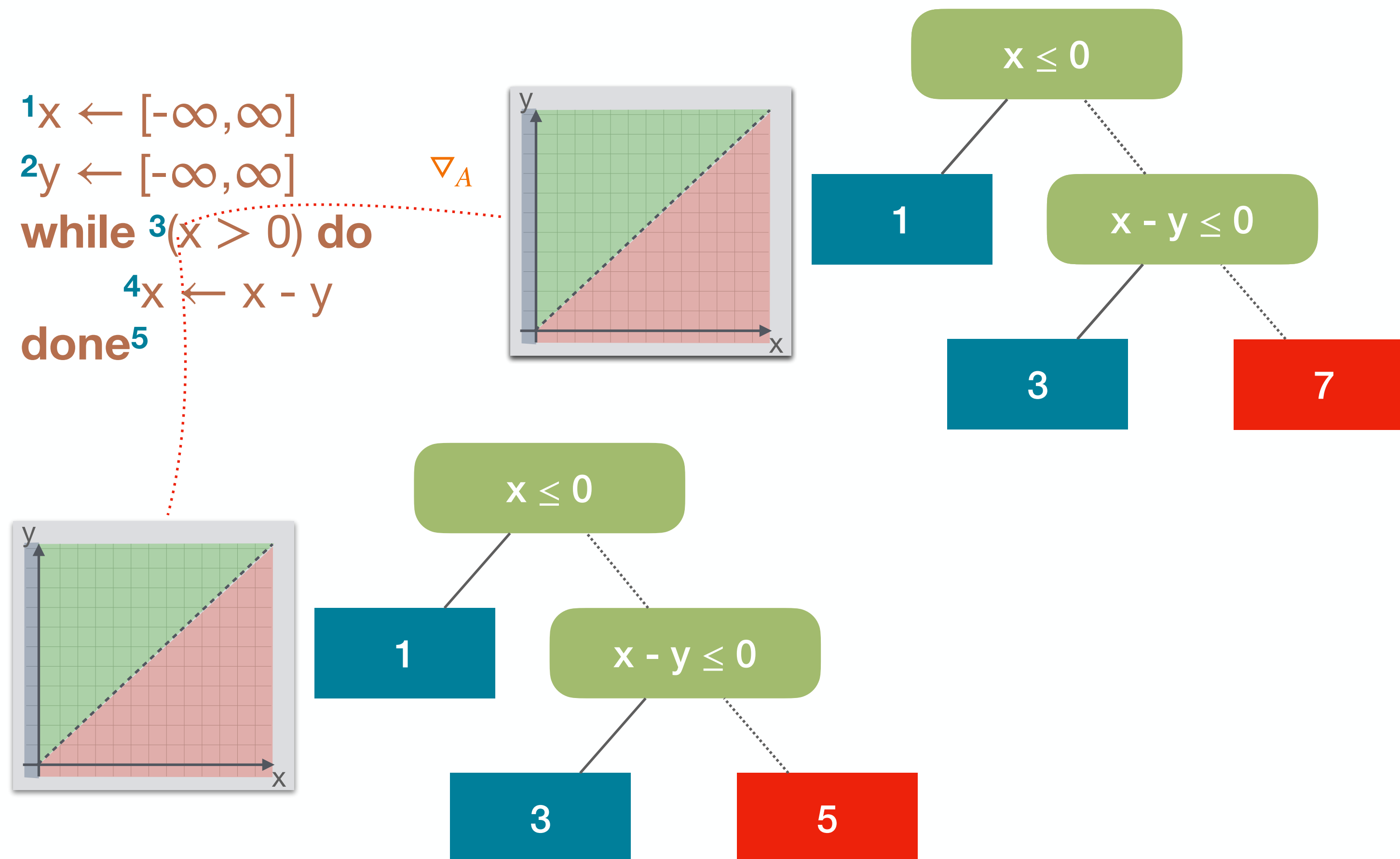
Abstract Definite Termination Semantics

Example



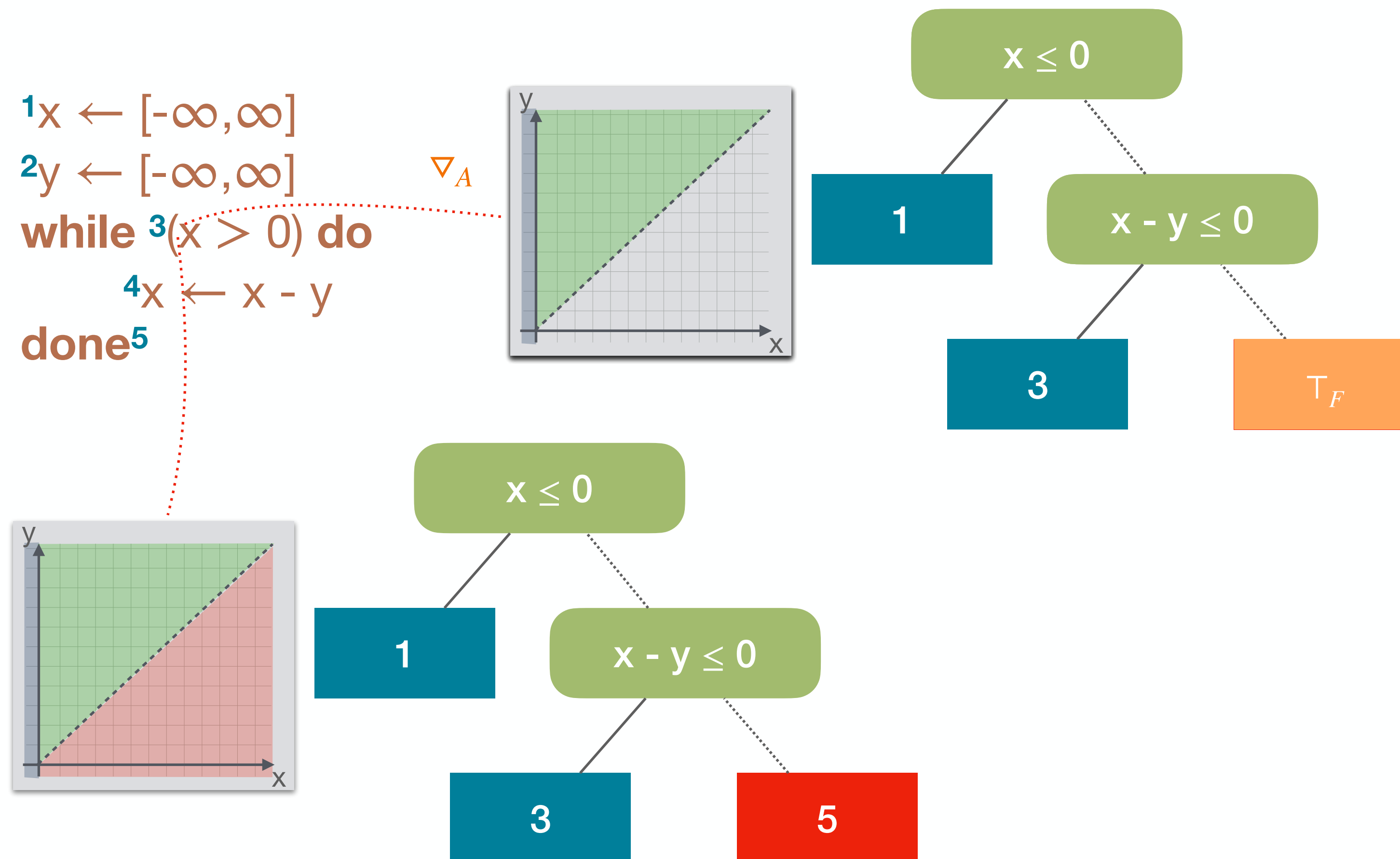
Abstract Definite Termination Semantics

Example



Abstract Definite Termination Semantics

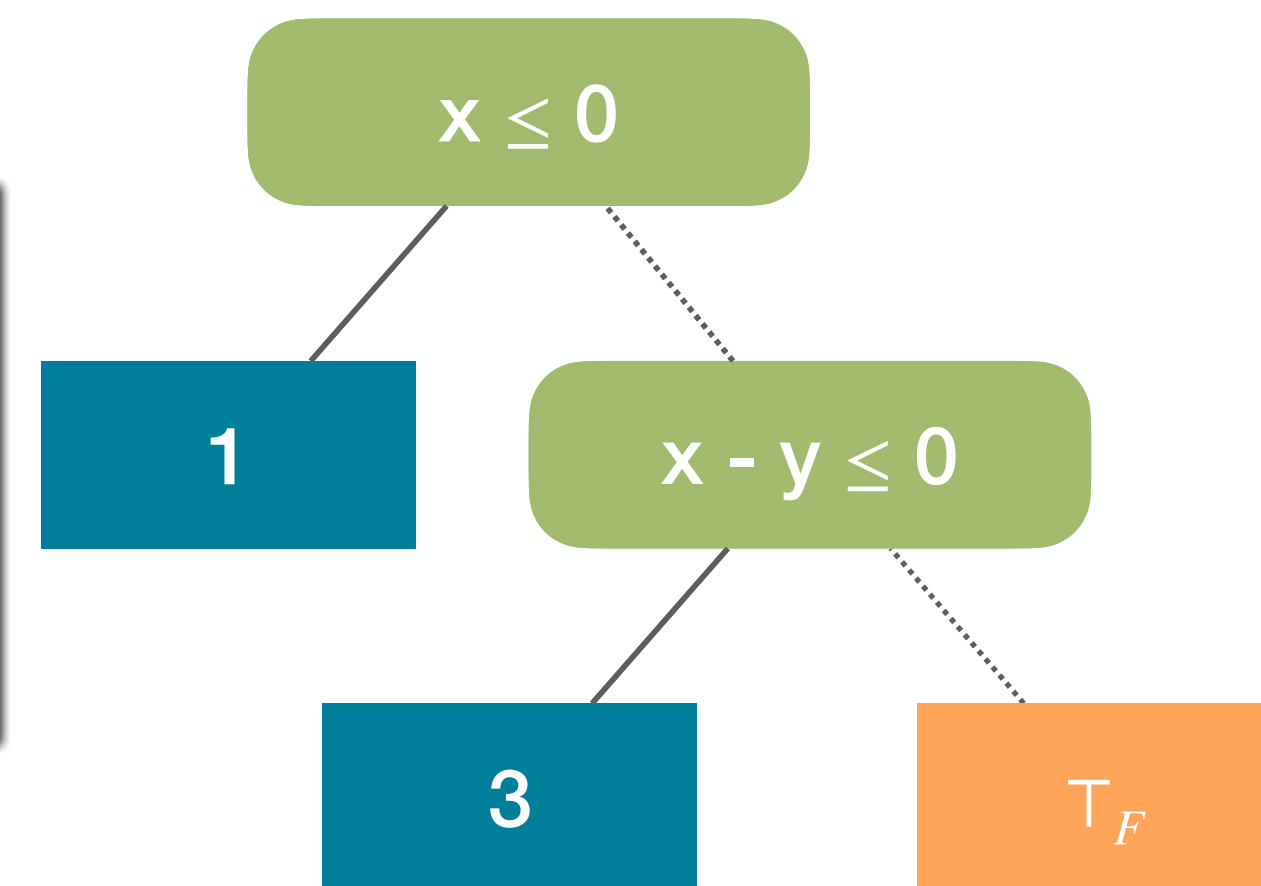
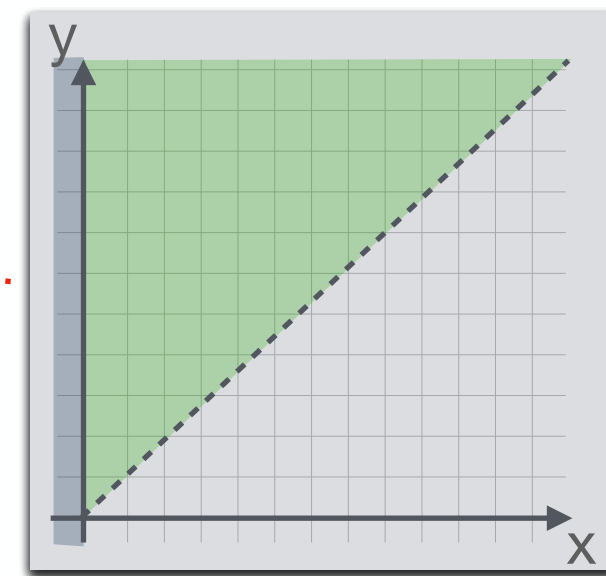
Example



Abstract Definite Termination Semantics

Example

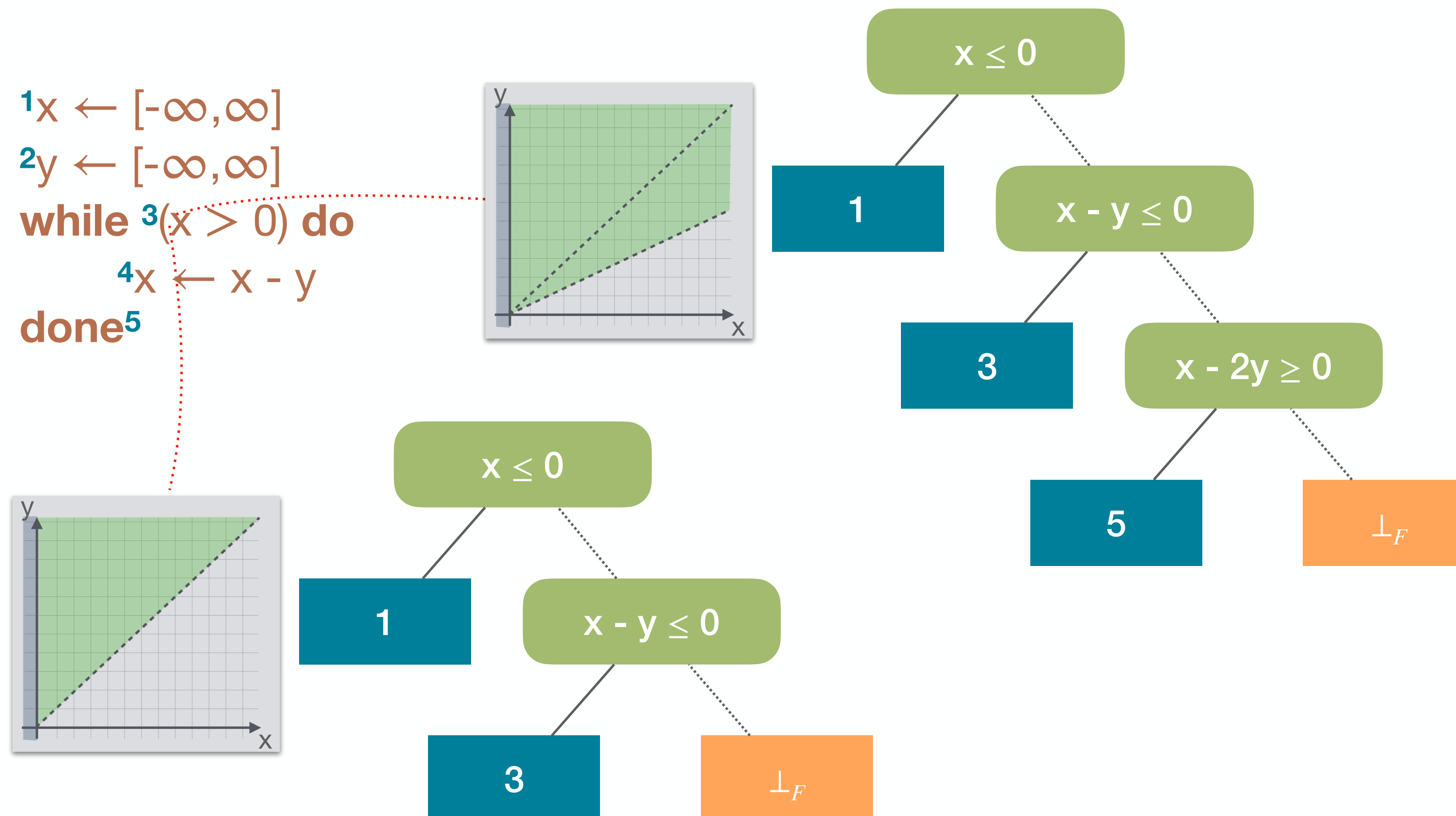
```
1  $x \leftarrow [-\infty, \infty]$   
2  $y \leftarrow [-\infty, \infty]$   
while 3  $(x > 0)$  do  
    4  $x \leftarrow x - y$   
done 5
```



Abstract Definite Termination Semantics

Example

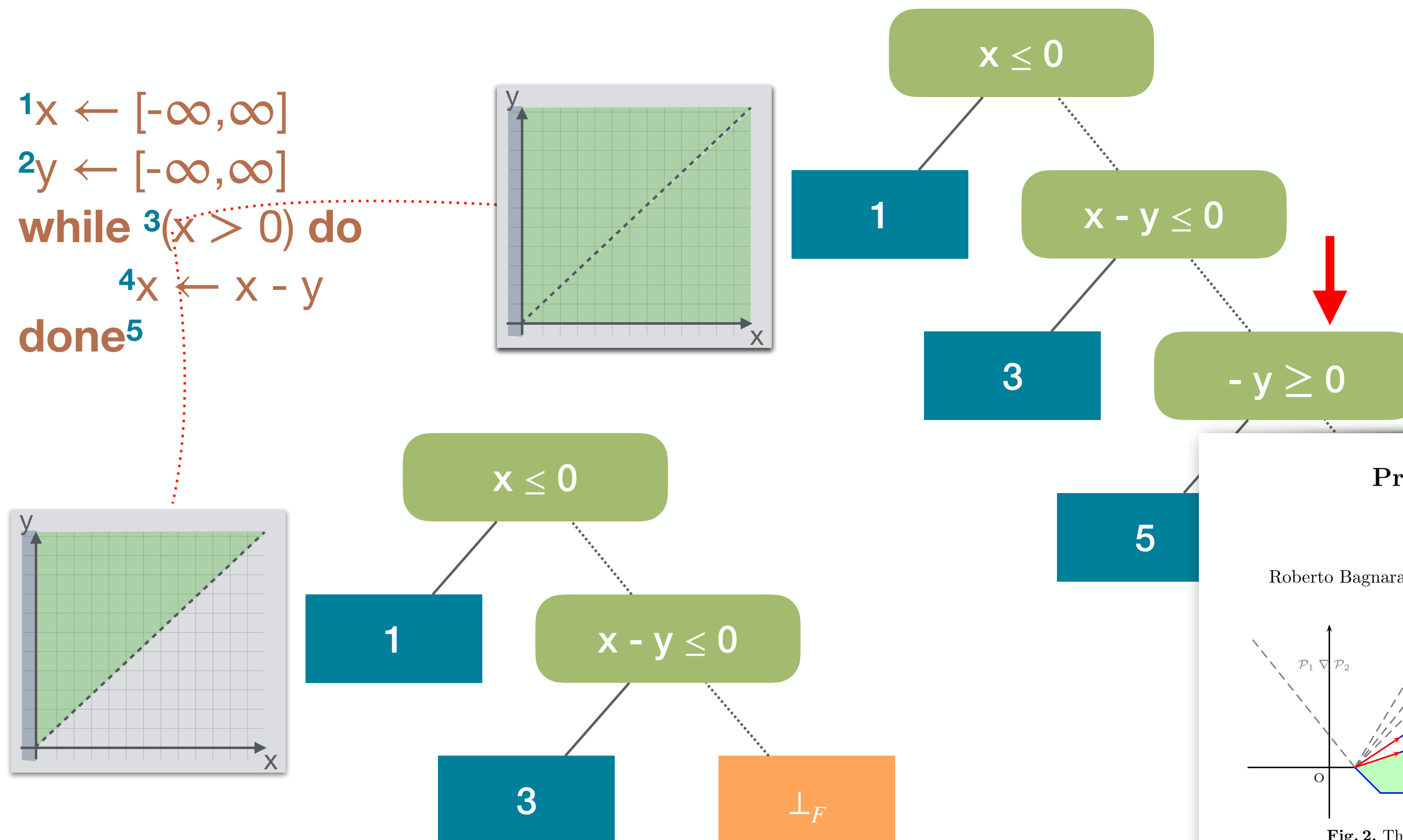
Better Widening



Abstract Definite Termination Semantics

Example

Better Widening



Precise Widening Operators for Convex Polyhedra*

Roberto Bagnara¹, Patricia M. Hill², Elisa Ricci¹, and Enea Zaffanella¹

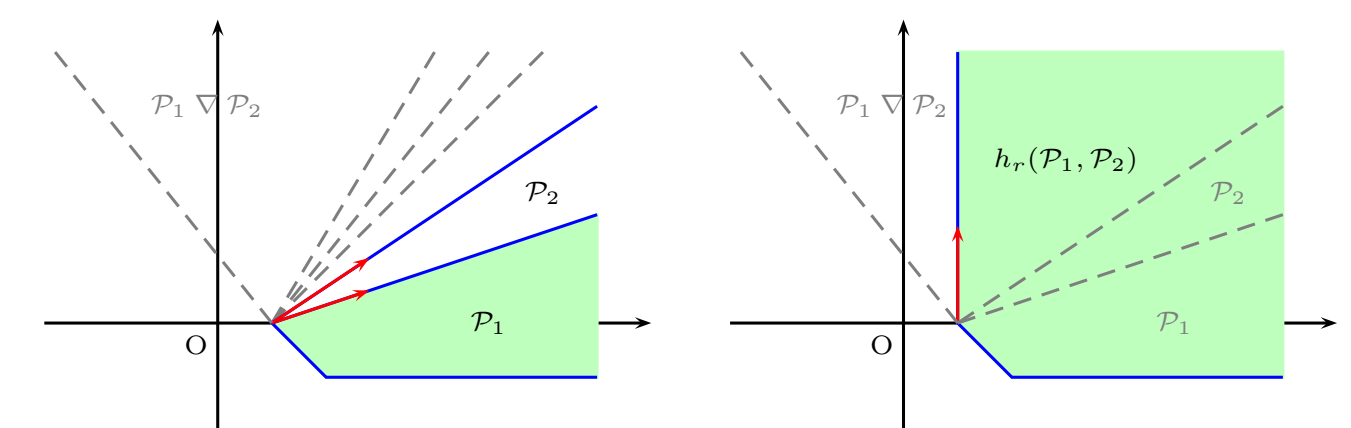
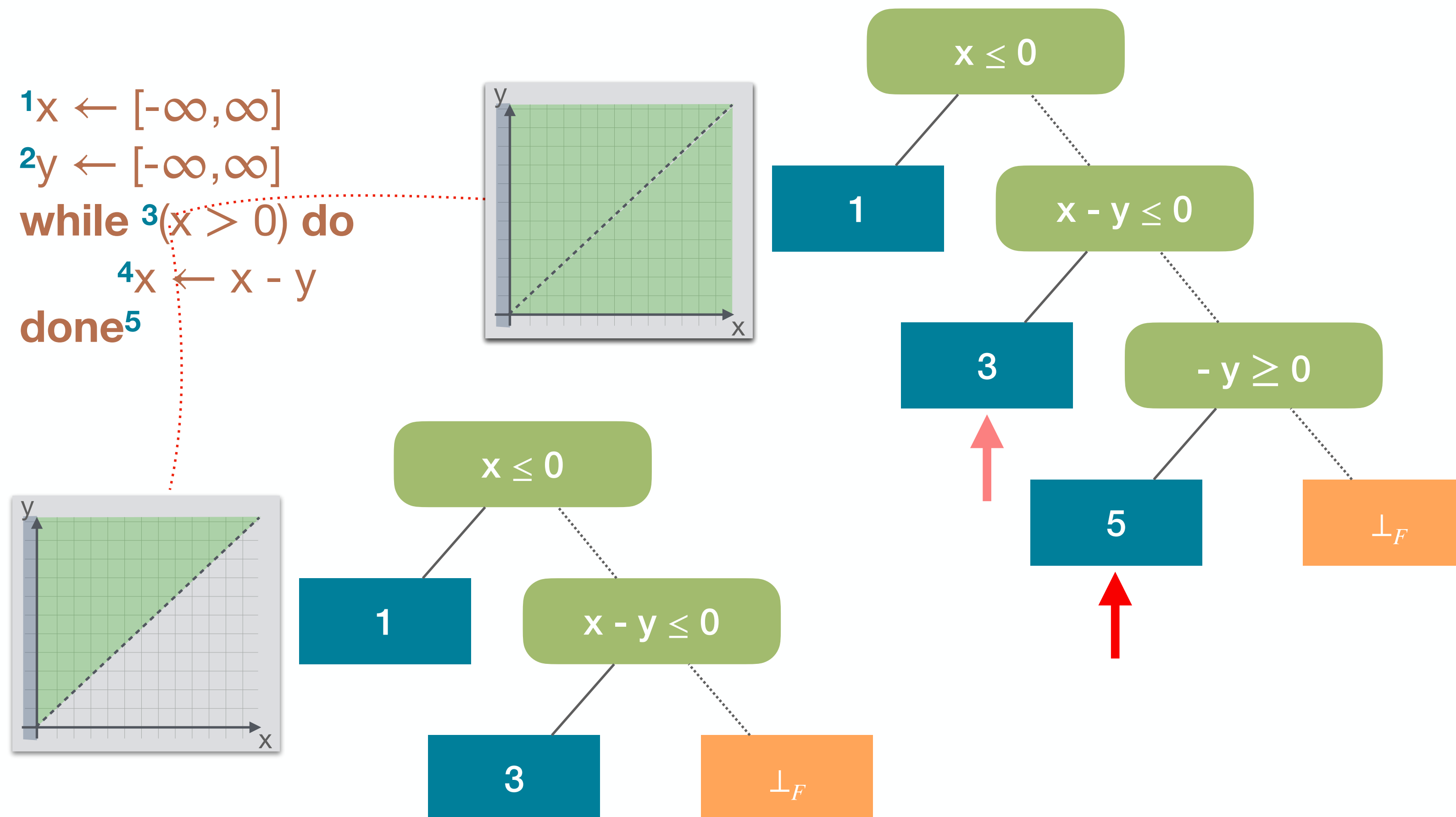


Fig. 2. The heuristics h_r improving on the standard widening.

Abstract Definite Termination Semantics

Example

Better Widening

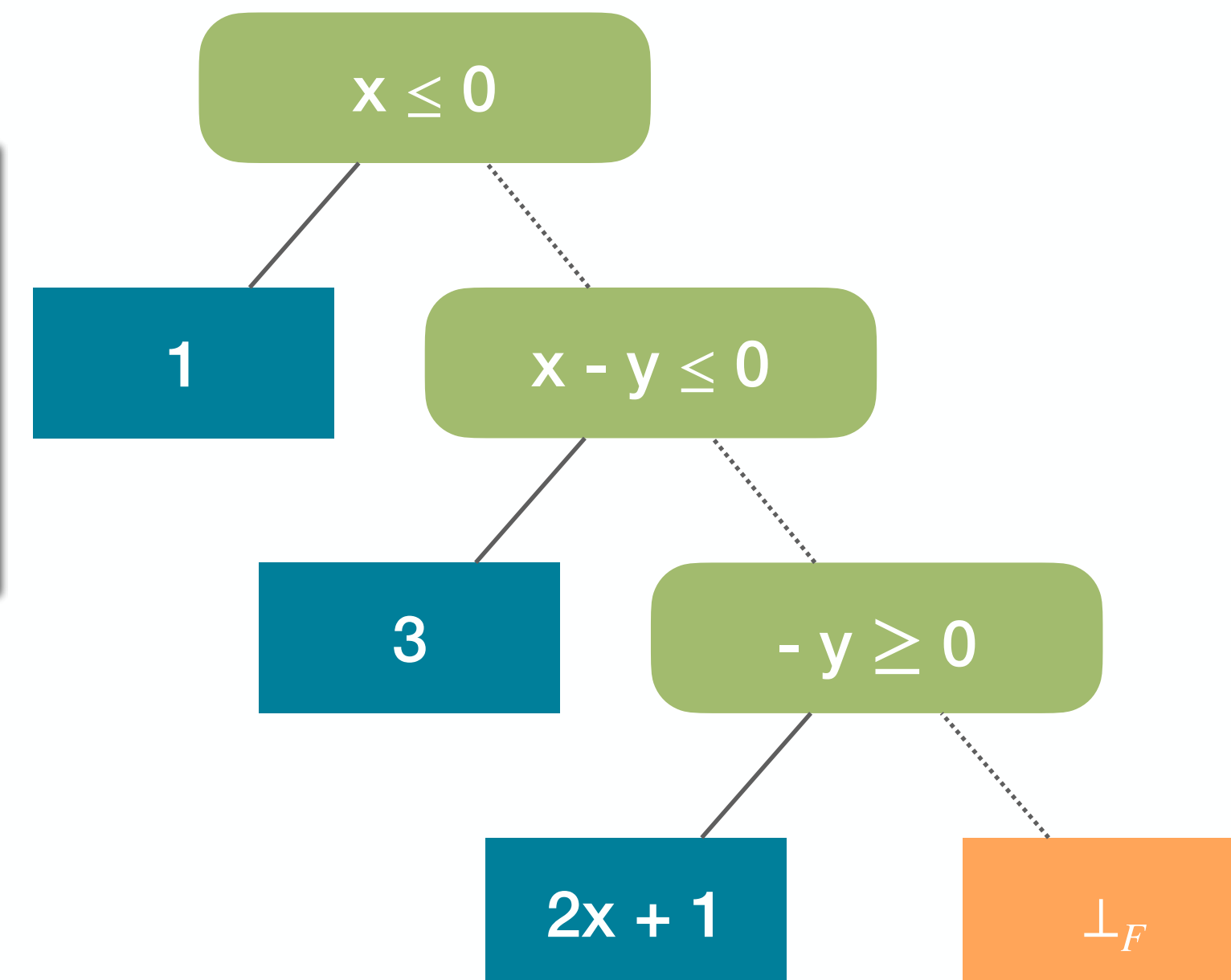
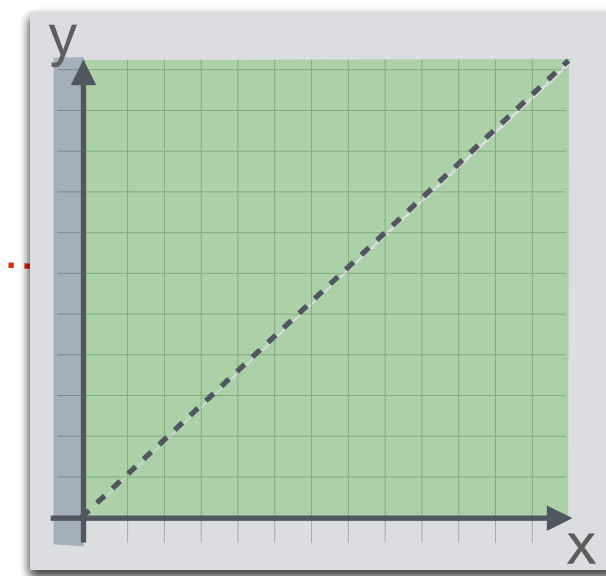


Abstract Definite Termination Semantics

Example

Better Widening

```
1  $x \leftarrow [-\infty, \infty]$   
2  $y \leftarrow [-\infty, \infty]$   
while 3  $(x > 0)$  do  
    4  $x \leftarrow x - y$   
done 5
```

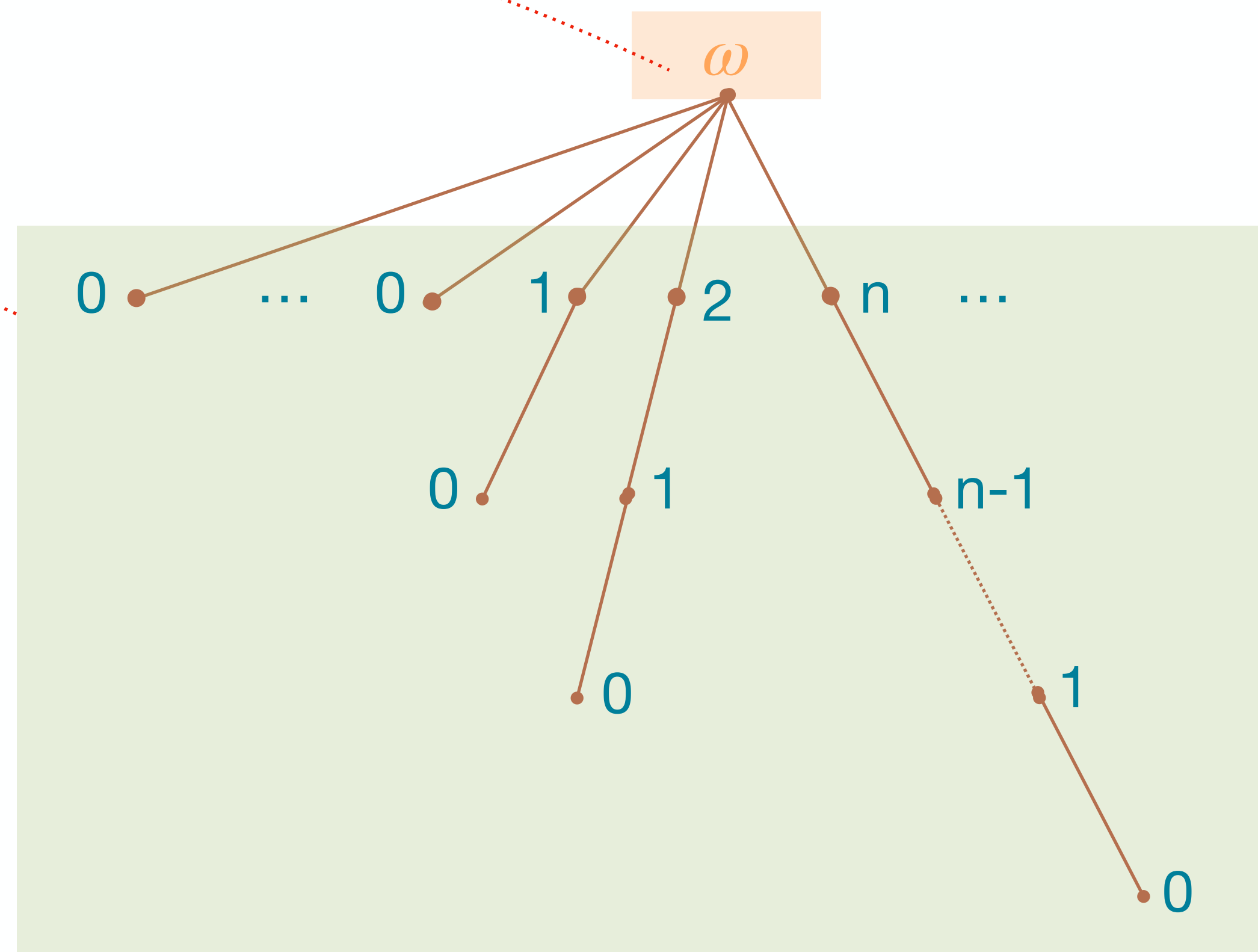


Ordinal-Valued Ranking Functions

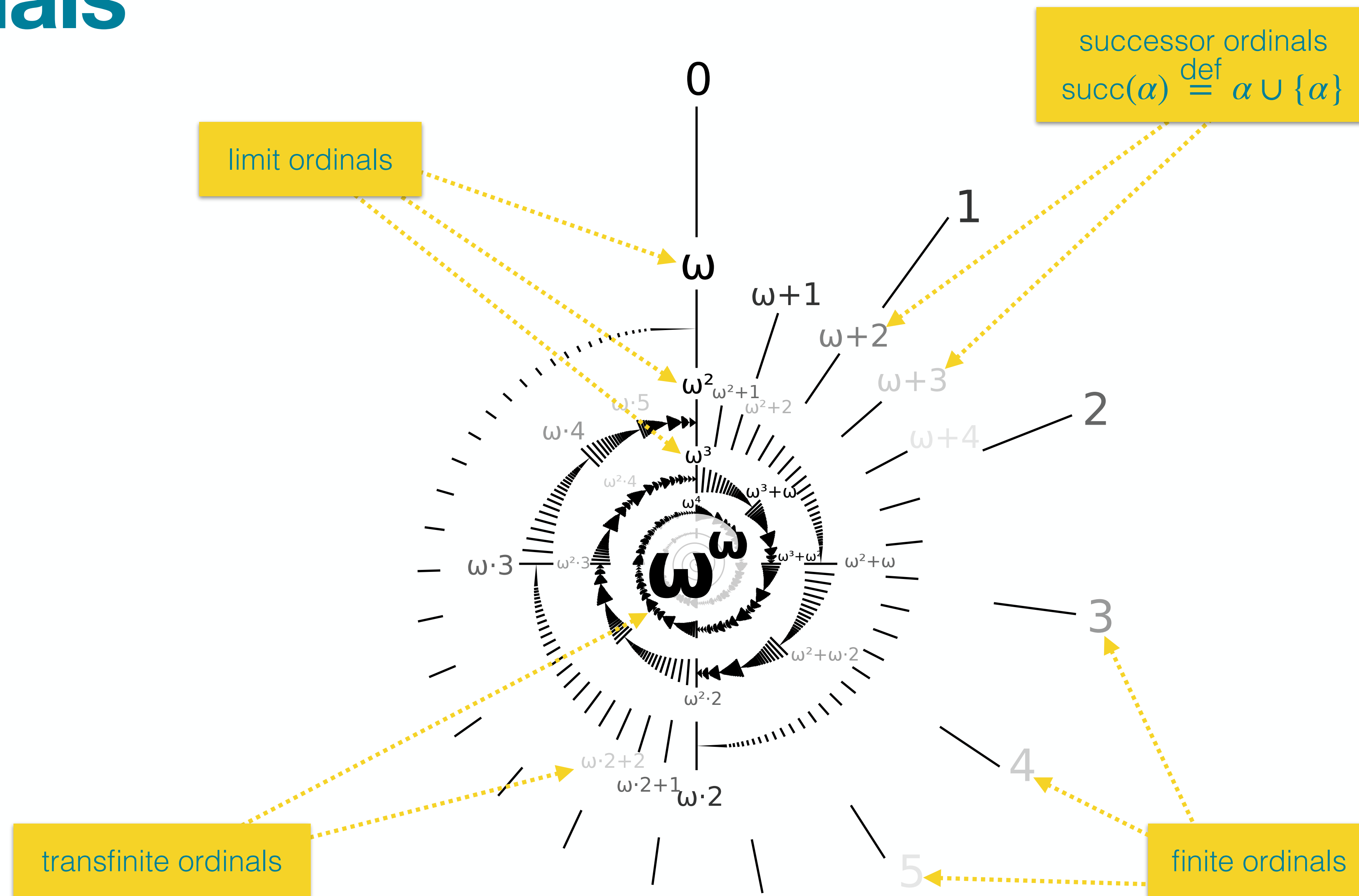
Need for Ordinals

Example

```
1 $x \leftarrow [-\infty, +\infty]$   
while 2 $(x > 0)$  do  
  3 $x \leftarrow x - 1$   
done4
```



Ordinals



Piecewise-Defined Function Domain

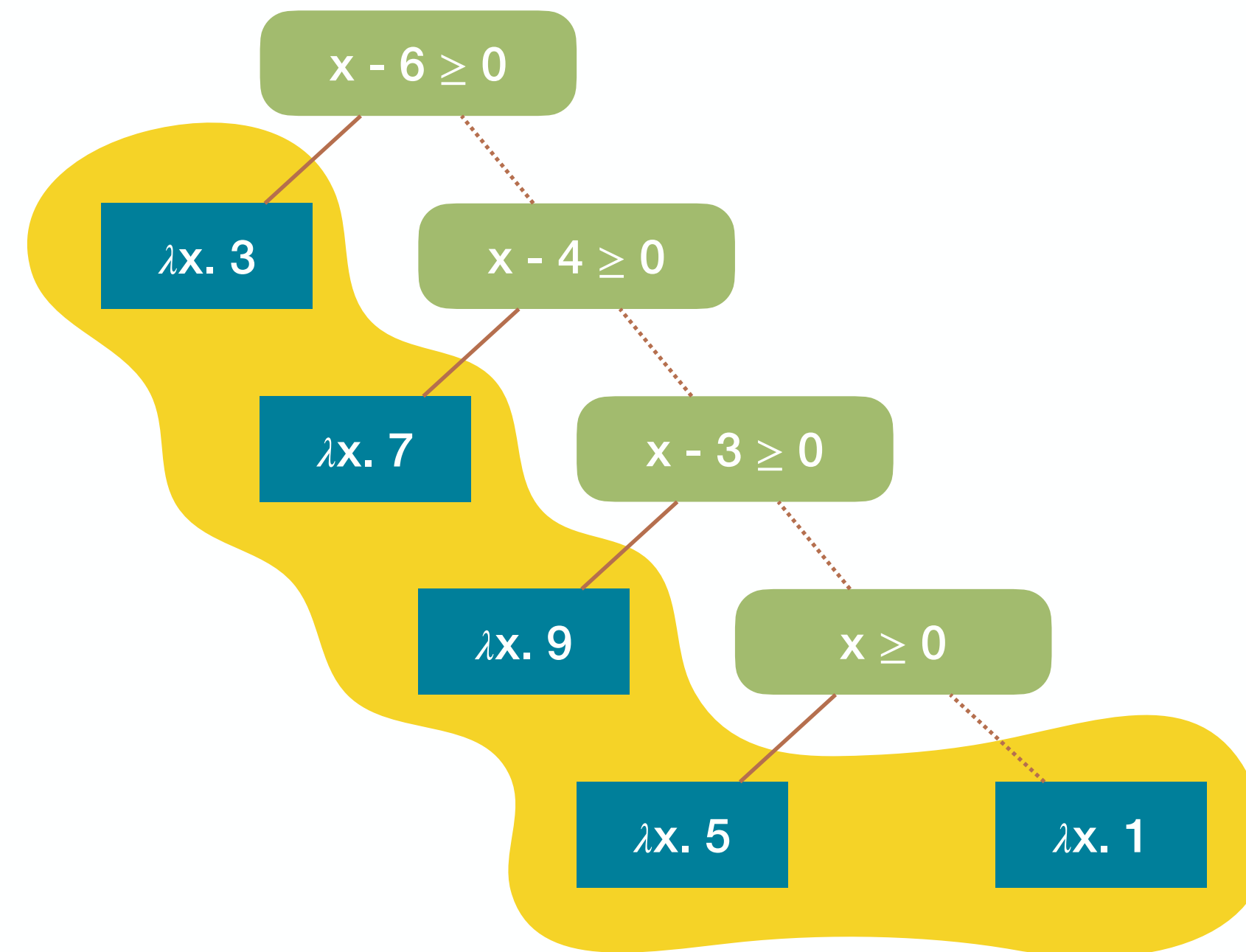
Functions Auxiliary Abstract Domain

- Parameterized by an *underlying numerical abstract domain* $\langle \mathcal{D}, \sqsubseteq_D \rangle$

- $\mathcal{W} \stackrel{\text{def}}{=} \{ \perp_W \} \cup \{ \sum_i \omega^i \cdot f_i \mid f_i \in \mathcal{F} \setminus \{ \perp_F, \top_F \} \} \cup \{ \top_W \}$

Cantor Normal Form
 $\omega^{\beta_1} \cdot n_1 + \dots + \omega^{\beta_k} \cdot n_k$

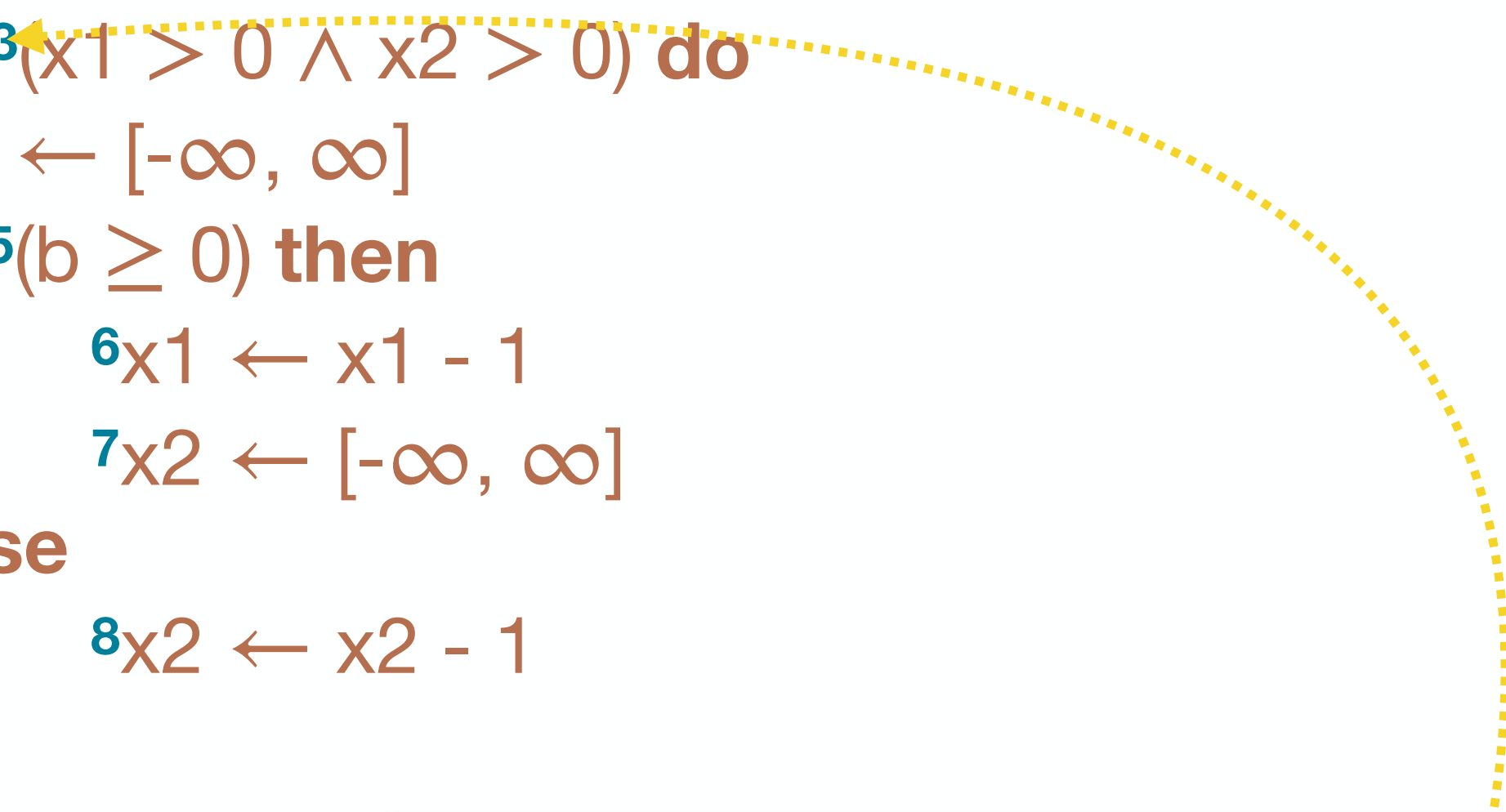
- $\mathcal{F} \stackrel{\text{def}}{=} \{ \perp_F \} \cup (\mathbb{Z}^{|V|} \rightarrow \mathbb{N}) \cup \{ \top_F \}$



Abstract Definite Termination Semantics

Example

```
1  $x_1 \leftarrow [-\infty, \infty]$   
2  $x_2 \leftarrow [-\infty, \infty]$   
3 while  $(x_1 > 0 \wedge x_2 > 0)$  do  
    4  $b \leftarrow [-\infty, \infty]$   
    if  $(b \geq 0)$  then  
        6  $x_1 \leftarrow x_1 - 1$   
        7  $x_2 \leftarrow [-\infty, \infty]$   
    else  
        8  $x_2 \leftarrow x_2 - 1$   
9 done
```



$$f_3 \stackrel{\text{def}}{=} \begin{cases} 1 & x_1 \leq 0 \vee x_2 \leq 0 \\ \omega \cdot (x_1 - 7) + 7x_1 + 3x_2 - 5 & x_1 > 0 \wedge x_2 > 0 \end{cases}$$

Static Termination Analysis

FuncTion

practical tools

targeting specific programs



abstract semantics, abstract domains

algorithmic approaches to decide program properties

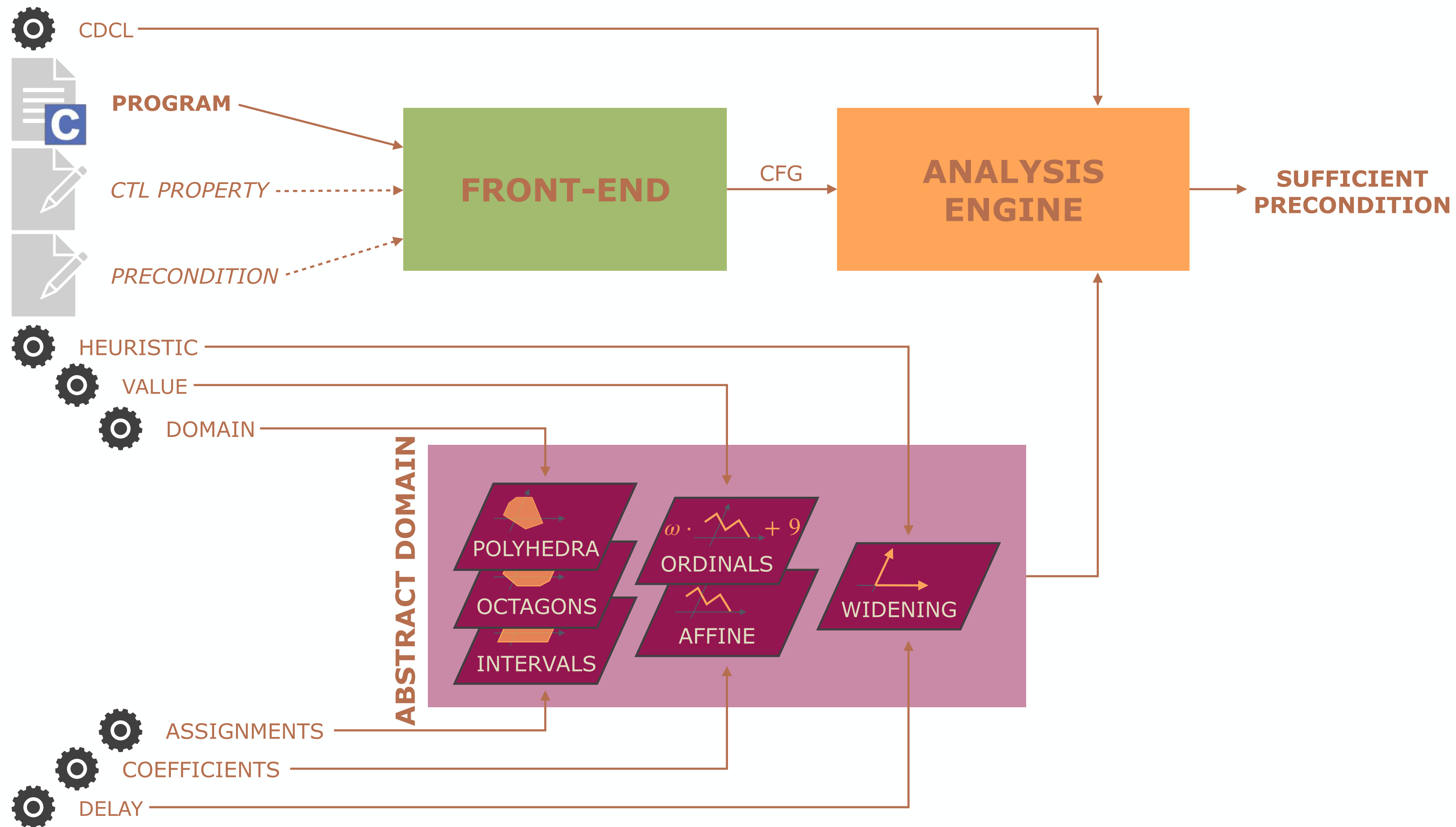


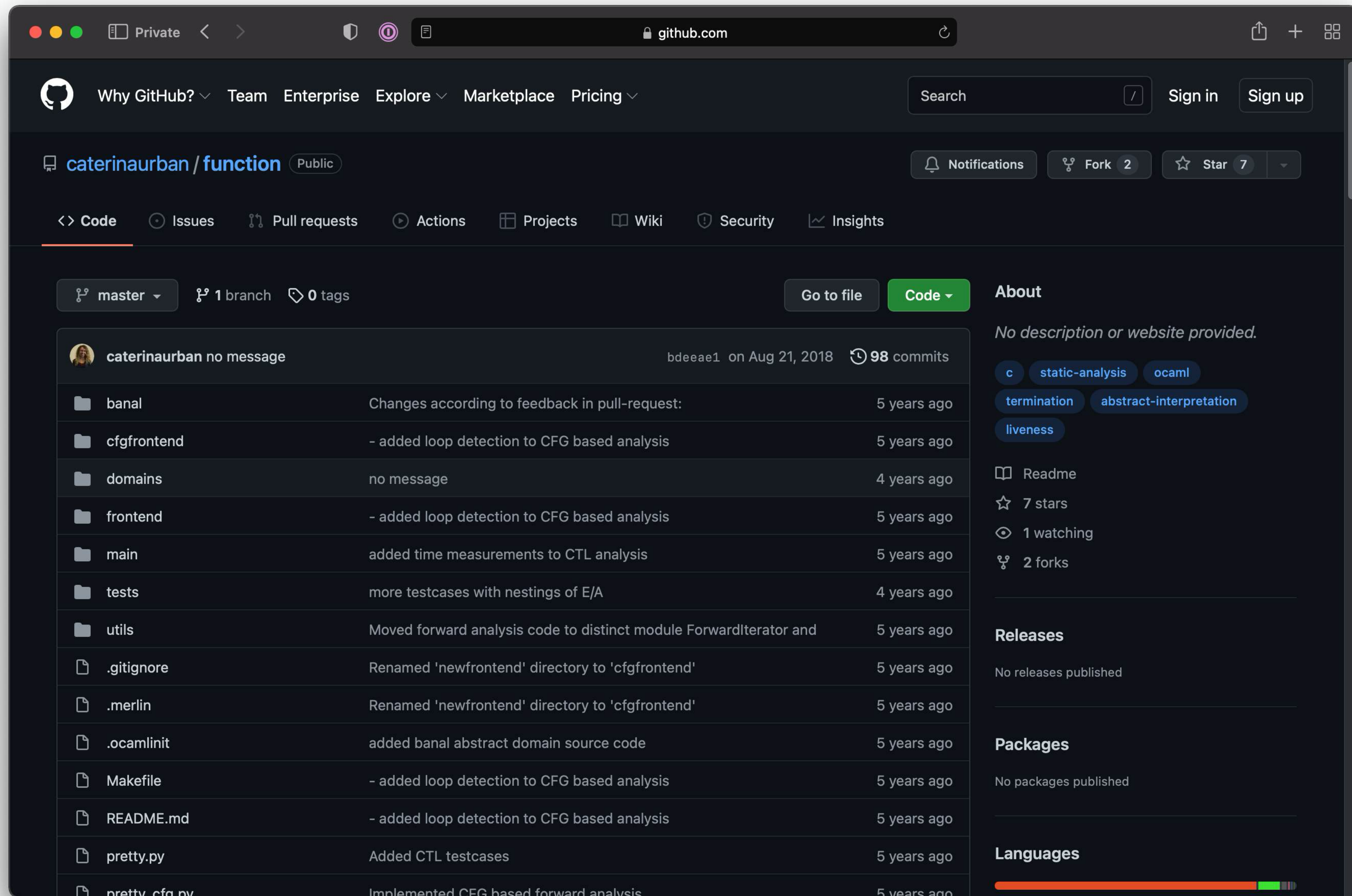
concrete semantics

mathematical models of the program behavior



FuncTion





Liveness Properties

Liveness Properties

- **Guarantee Properties**
“something good eventually happens at least once”
 - Example: Program Termination
- **Recurrence Properties**
“something good eventually happens infinitely often”
 - Example: Starvation Freedom



Zohar Manna

Amir Pnueli

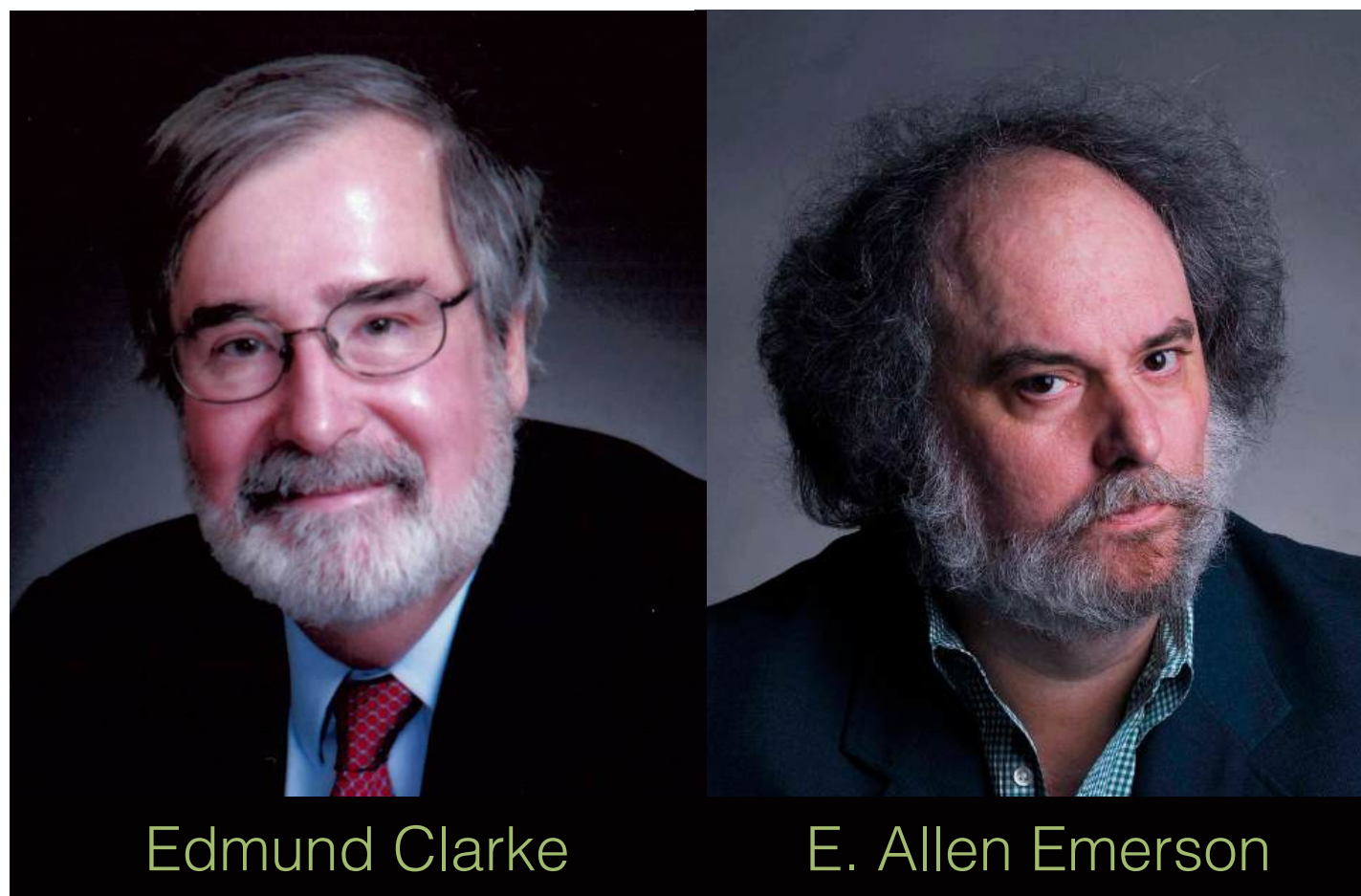
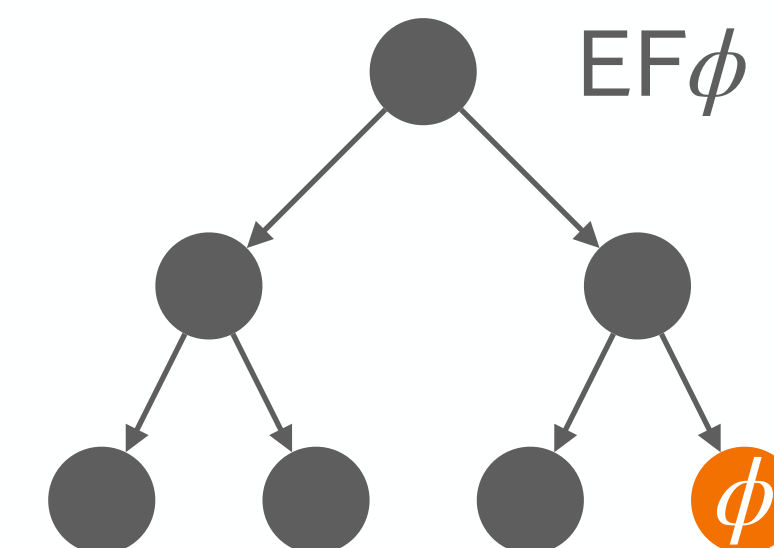
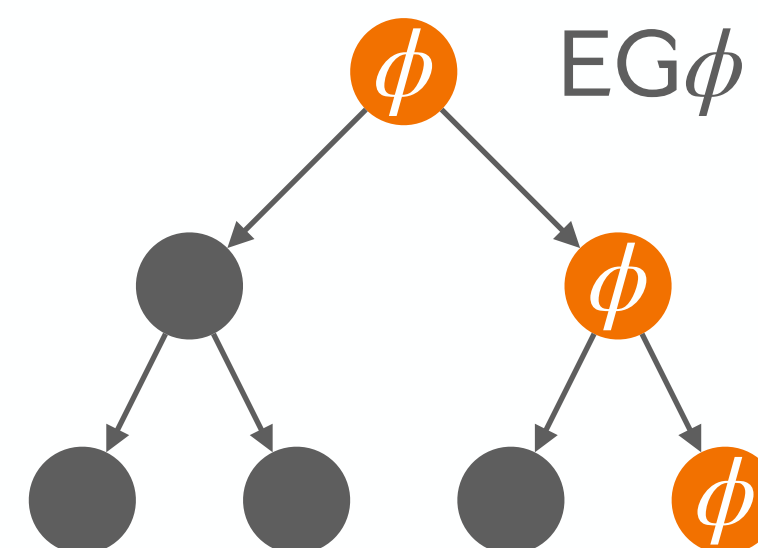
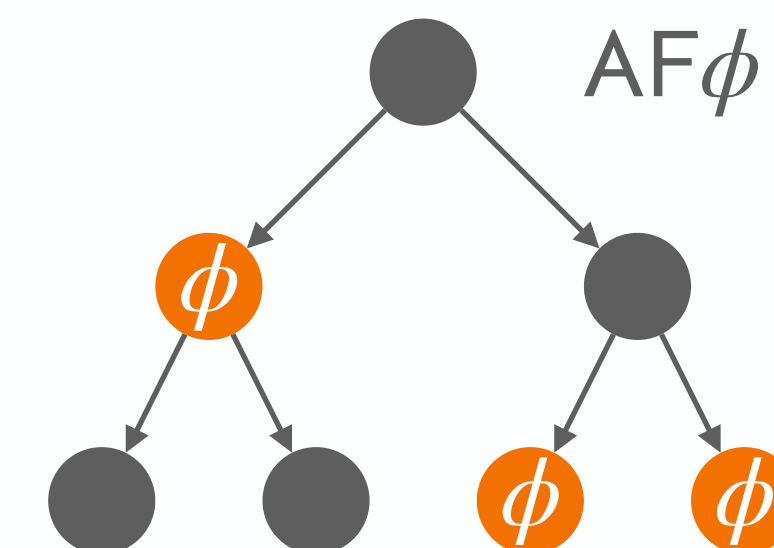
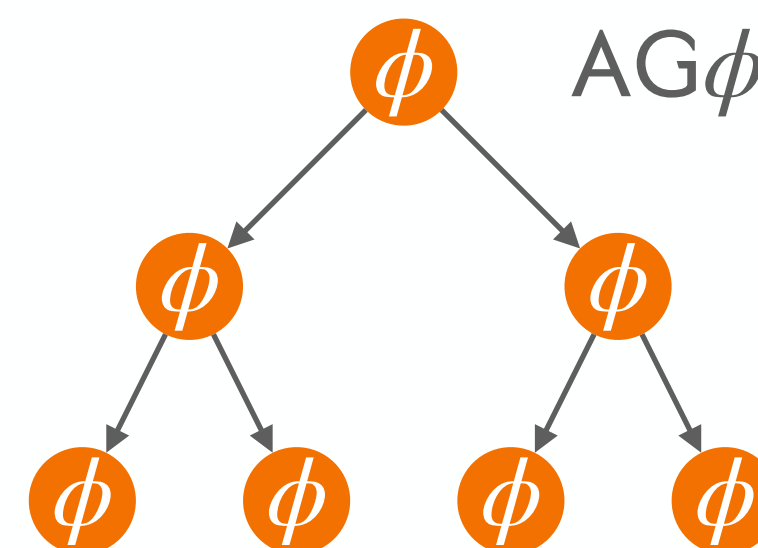
Computation Tree Logic (CTL)

Branching Temporal Logic

$\phi ::= a \mid \neg\phi \mid \phi \wedge \phi \mid \phi \vee \phi \mid AX\phi \mid AG\phi \mid A(\phi U \phi) \mid EX\phi \mid EG\phi \mid E(\phi U \phi)$

$AF\phi \equiv A(\text{true} U \phi)$

$EF\phi \equiv E(\text{true} U \phi)$



Reading Suggestion

Abstract Interpretation of CTL Properties

Caterina Urban, Samuel Ueltschi, and Peter Müller

Department of Computer Science
ETH Zurich, Switzerland



Abstract. CTL is a temporal logic commonly used to express program properties. Most of the existing approaches for proving CTL properties only support certain classes of programs, limit their scope to a subset of CTL, or do not directly support certain existential CTL formulas. This paper presents an abstract interpretation framework for proving CTL properties that does not suffer from these limitations. Our approach automatically infers sufficient preconditions, and thus provides useful information even when a program satisfies a property only for some inputs. We systematically derive a program semantics that precisely captures CTL properties by abstraction of the operational trace semantics of a program. We then leverage existing abstract domains based on piecewise-defined functions to derive decidable abstractions that are suitable for static program analysis. To handle existential CTL properties, we augment these abstract domains with under-approximating operators. We implemented our approach in a prototype static analyzer. Our experimental evaluation demonstrates that the analysis is effective, even for CTL formulas with non-trivial nesting of universal and existential path quantifiers, and performs well on a wide variety of benchmarks.